

KV Cache Compression Through the Lens of Transform Coding

Hannah Laus, Claudio Mayrink Verdun, Hao Wang, Flavio du Pin Calmon, Felix Krahmer

Abstract—The key-value (KV) cache stores information from past tokens and is a major memory bottleneck in long-context inference. Existing quantization methods address this bottleneck by representing the KV cache uniformly with lower-precision data types and designing quantization schemes to minimize reconstruction error in the cache itself, without accounting for how that error propagates through attention mechanisms. We prove that, under a white-noise quantization model, the expected attention-aware distortion decomposes into additive key and value contributions that factor across tokens and channels. Building on transform coding and reverse water-filling, which are classical tools from signal processing and rate-distortion theory, we introduce Attention-Aware Transform Coding (AATC), which allocates bits over a calibration set to minimize attention-aware distortion. On Llama-3.1-8B-Instruct and Qwen-2.5-7B-Instruct, evaluated across LongBench, RULER, GSM8K, MMLU-Pro, and MATH-500, our method achieves near-lossless accuracy at approximately $5.8\times$ compression, whereas each baseline degrades in at least some settings.

Index Terms—KV cache compression, quantization, transform coding, reverse waterfilling, large language models

I. INTRODUCTION

Signal quantization, that is, the representation of signals by bit sequences, is a key building block of digital signal processing. A central task in this context is bit allocation, the question of how many bits to use for different parts of the signal representation. This topic is well studied in the signal processing literature and in various models, one can precisely characterize how to optimally allocate the bits (see [1] for a detailed discussion).

Key ingredients for many methods attempting optimal bit allocation are transform coding [2] and reverse water-filling [3]. The transform coding step whitens the signal, yielding a signal representation with components that are uncorrelated but of varying strengths. The reverse water-filling step then distributes the bits to best represent the components under a fixed budget, allocating more bits to larger components and fewer bits to smaller components.

In recent years, an analogous problem of data representation with a fixed bit budget has arisen in machine

learning in the context of inference with large language models (LLMs). At inference time, an LLM decomposes its input into tokens and then generates new tokens by sampling from next token predictions. In transformer models [4], each token yields a sequence of key-value (KV) pairs at each layer of the model. These values are cached to accelerate inference when generating new tokens. At the same time, LLMs increasingly operate over long contexts: documents, transcripts, and multi-turn histories spanning more than a million tokens. As a consequence, the KV cache grows linearly with context length, and at long contexts it becomes the dominant memory cost of inference. KV cache compression has become essential for deploying transformer-based LLMs under realistic memory and latency budgets. In that context, efficient quantization plays a key role [5], [6]. Approaches based on low-rank projection [7], [8] can be interpreted as identifying components with no bits allocated to them. A third ingredient in state-of-the-art KV cache compression that is somewhat orthogonal is based on eviction [9], [10].

Quantization and low-rank projection are highly effective for KV cache compression, and several such methods already draw on ideas from signal quantization and transform coding [7], [8], [11]. We develop this connection systematically, studying KV cache compression through the lens of transform coding. This perspective leads us to an attention-aware distortion measure that, rather than minimizing raw reconstruction error on the cache, decomposes the impact of quantization on the output into three distinct error dimensions, which have been individually studied in previous works, but without including their interaction. The decomposition unifies several previously disconnected viewpoints: it shows that these separately-discovered effects in token eviction, quantization or low-rank projection are facets of a single distortion objective, which both motivates our method and provides a unifying perspective on existing work that we map onto the individual factors in Section IV-B.

To distribute the bits in a way that leads to a small distortion, we determine a bit allocation scheme via a reverse waterfilling algorithm [3] applied to calibration data and then apply it individually to each token. Inspired by the fact that reverse waterfilling is designed for uncorrelated data and is therefore often combined with a transform coding step for decorrelation [2], our method first whitens the key and value features on a calibration set, which approximately decorrelates them and transforms the data into components of varying strength, before distributing the total bit budget across the transformed channels by reverse waterfilling. We validate these findings on Llama-3.1-8B-Instruct and Qwen-2.5-7B-Instruct,

Hannah Laus and Felix Krahmer are with the Technical University of Darmstadt, the Technical University of Munich and the Munich Center for Machine Learning (hannah.laus@tum.de)

Claudio Mayrink Verdun is with the Massachusetts Institute of Technology

Hao Wang is with the Red Hat AI Innovation Team and the MIT-IBM Watson AI Lab

Flavio du Pin Calmon is with the John A. Paulson School Of Engineering And Applied Sciences at Harvard University

across LongBench, RULER, GSM8K, MMLU-Pro, and MATH-500 which are standard long-context quality benchmarks.

Our main contributions are as follows.

- We derive an attention-aware distortion measure that separates key and value quantization errors into token- and channel-dependent factors.
- We provide a unifying perspective across several existing KV cache compression methods by relating them to the attention-aware distortion measure.
- Based on this measure, we develop Attention-Aware Transform Coding (AATC), which combines whitening with reverse waterfilling to obtain an optimal channel-wise allocation under the attention-aware distortion.
- We show that AATC achieves near-lossless accuracy across all benchmarks at approximately $5.8\times$ compression, outperforming the baselines (KIVI, KVQuant and PALU), since they degrade at least on one benchmark.

II. RELATED WORK

A. KV cache Compression

Approaches to reducing the memory footprint of the KV cache fall into several families. Scalar quantization methods reduce per-element precision: KIVI [5] uses per-channel asymmetric quantization for keys and per-token quantization for values; KVQuant [6] employs non-uniform quantization with sensitivity-weighted codebooks and dense outlier handling. These methods preserve all dimensions of the cache and degrade precision uniformly or near-uniformly. Vector quantization methods such as A²ATS [12], CommVQ [13], and TurboQuant [14] jointly quantize groups of coefficients via learned codebooks. Low-rank and transform-based methods, such as PALU [7], Eigen Attention [15] and xKV [16] compress along the feature dimension by deleting low-energy subspaces from the cache. Transform-coding methods instead decorrelate and allocate bits without hard truncation: KVTC [8] applies a global, cross-layer transform followed by bit allocation and is the closest prior method to ours. MiniCache [17] alternatively merges KV pairs across layers. Rotation-based methods [18], [19] apply orthogonal transforms before quantization to suppress outliers. A complementary line of work [20], [21] aims to minimize the distortion after multiplying key and query vectors, as this corresponds to the input of the attention.

Eviction and merging methods reduce the cache by combining or deleting previous tokens: H2O [9], SnapKV [22] and StreamingLLM [10] evict tokens based on different importance scoring techniques, which are all related to attention-derived importance. A more recent line of eviction work [23]–[26] refines the importance criterion beyond accumulated attention scores by incorporating value-side information. The new eviction criteria mentioned in those works correspond to projections of our decomposition onto its token-dependent factor (Section IV-B). Our work is in the transform-coding family and proposes a bit allocation

criterion derived from a general distortion analysis. We perform empirical comparisons against KIVI, PALU, and KVQuant in Section VI. In Appendix C we relate many of the above works to our distortion decomposition (Theorem 1).

B. Theoretical Analyses of KV cache Quantization

Several recent works analyze KV cache quantization theoretically, each tied to a specific method. Closest to our key-side analysis is QAQ [27], which via a short second-order expansion obtains the query-weighted key sensitivity and allocates per-token bits from the aggregate query norm. We instead resolve the query weighting per channel, carry a controlled remainder (Lemma 1), and add the value-side weight. Concurrent work [28] likewise splits the attention-output error into key and value terms, but as a worst-case runtime bound used to fall back to exact attention rather than to allocate precision offline, so it carries no channel weights or transform. [29] applies random matrix theory, modeling the cache as a low-rank shared-context component plus a residual that is unique for every token. RateQuant [11] is based on applying rate-distortion theory to head-level mixed-precision allocation. Finally, Zhou et al. [19] prove that attention-aware rotations are optimal under a frozen residual model, with the per-channel bit allocation held uniform rather than optimized. We further relate the last two papers to our distortion decomposition in Section IV-B and Appendix C.

C. Signal Quantization

On the signal-processing side, our method instantiates the classical transform-coding architecture, first decorrelate, then allocate, introduced by Huang and Schultheiss [30], whose components we review in Section III-A. Closest in spirit is perceptual coding [31], which adapts the distortion measure to the receiver of the signal, there the human perceptual system. We adapt it to the transformer’s read-out. Here attention plays the role of the perceptual model, with the induced weighted metric derived in Theorem 1 rather than posited. Learned nonlinear transform coding [32] continues the same blueprint with trained nonlinear transforms. In contrast, ours is fixed by calibration statistics, so the transform is exactly invertible and any loss is confined to the allocation step.

III. PRELIMINARIES

We briefly review transform coding, attention and KV cache quantization.

A. Transform Coding

Let $\mathbf{x} \in \mathbb{R}^d$ be a zero-mean source with covariance $\Sigma \in \mathbb{R}^d \times \mathbb{R}^d$, and let $\hat{\mathbf{x}}$ be its reconstruction after lossy compression. We measure fidelity by the average squared-error distortion per dimension [1],

$$D = \frac{1}{d} \mathbb{E}[\|\mathbf{x} - \hat{\mathbf{x}}\|_2^2] = \frac{1}{d} \sum_{i=1}^d D_i, \quad D_i = \mathbb{E}[(x_i - \hat{x}_i)^2]. \quad (1)$$

When each coordinate is quantized separately, the design problem is thus to distribute a bit budget across coordinates.

Transform coding [2], the framework behind classical media codecs such as JPEG [33], solves this problem in a transformed coordinate system: an orthonormal transform $\mathbf{y} = \mathbf{U}^\top \mathbf{x}$ decorrelates the source and concentrates its variance in a few leading coefficients (*energy compaction*); the coefficients are quantized with independent scalar quantizers, and the signal is reconstructed as $\hat{\mathbf{x}} = \mathbf{U}\hat{\mathbf{y}}$. Orthonormal transforms preserve the MSE, so (1) applies unchanged to the coefficients y_i , whose variances σ_i^2 now differ widely: high-variance coefficients deserve many bits, low-variance ones few or none. For Gaussian sources, the optimal transform is the Karhunen–Loève transform, the eigenbasis of Σ . In practice one uses a whitening transform estimated from data.

To allocate the bits, note that under high-rate assumptions a scalar quantizer with b_i bits achieves $D_i \approx c\sigma_i^2 2^{-2b_i}$ for a quantizer-dependent constant c [1]. Minimizing (1) subject to $\frac{1}{d}\sum_i b_i = \bar{b}$ and $b_i \geq 0$ yields the classical allocation of Huang and Schultheiss [30], which coincides with the reverse waterfilling solution for a source of independent Gaussian components [3, Chapter 10]:

$$b_i^* = \begin{cases} \frac{1}{2} \log_2 \frac{\sigma_i^2}{\lambda}, & \text{if } \sigma_i^2 > \lambda \\ 0 & \text{otherwise,} \end{cases} \quad (2)$$

where the water level λ is chosen so that $\frac{1}{d}\sum_i b_i^* = \bar{b}$. Every coefficient which is retained is quantized to the same distortion $c\lambda$, while coefficients whose variance falls below the water level receive no bits and are reconstructed as zero.

Both the transform and the allocation depend on source statistics that are typically unknown; in practice, they are estimated from a calibration sample, as we do in this work.

B. Attention and the KV cache

A decoder-only transformer [34], the architecture behind modern large language models, takes a sequence of tokens and predicts the next token. Internally, each token is represented as a vector and the transformer applies L layers in sequence, to map the current token vectors to new ones. Within a layer, the token vectors are first mixed across the sequence and then passed through a per-token nonlinear-map. The mixing step, called self-attention, is the only operation that introduces interdependence between the tokens.

In each layer the input is linearly projected into three sequences of vectors, the queries, keys and values. Intuitively, a token’s query states what information it is looking for, its key states what it offers and its value is the content it contributes. The self-attention mechanism maps each query to a data-dependent weighted combination across tokens. Unlike a fixed linear filter, the combination weights are computed from the input itself. More precisely, for every query, self-attention forms a scaled dot-product similarity with all keys, passes these scores through a

softmax to obtain nonnegative weights that sum to one, and returns the corresponding weighted average of the values.

To generate text, the model predicts the next token, appends it to the sequence and repeats. The sequence is the given context followed by the tokens generated so far, so its length T grows by one at every step and with it the set of keys and values that must remain available. To avoid recomputing them, the keys and values of all past tokens are stored in the KV cache, whose size therefore grows with T .

To represent this in mathematical notation, let T be the number of tokens and L the number of layers. The input to layer ℓ is $\mathbf{X}^{(\ell)} \in \mathbb{R}^{T \times d_{\text{model}}}$ whose t -th row $x_t^{(\ell)} \in \mathbb{R}^{d_{\text{model}}}$ is the hidden representation of token t at layer ℓ . Suppressing ℓ the query, key and value sequences introduced above are $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_K$, $\mathbf{V} = \mathbf{X}\mathbf{W}_V$ with $\mathbf{W}_Q^{(\ell)}, \mathbf{W}_K^{(\ell)} \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $\mathbf{W}_V^{(\ell)} \in \mathbb{R}^{d_{\text{model}} \times d_v}$ (typically $d_k = d_v = d_{\text{model}}$) and rows $q_t = x_t \mathbf{W}_Q, k_t, v_t$.

Definition 1 (Scaled Dot-Product Attention [4]). *Given $\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{T \times d_k}$, $\mathbf{V} \in \mathbb{R}^{T \times d_v}$, and a mask $\mathbf{M} \in (\mathbb{R} \cup \{-\infty\})^{T \times T}$,*

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) := \mathbf{A}\mathbf{V}\mathbf{W}_O, \quad \mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right),$$

with row-wise softmax weights

$$a_{ti} = \frac{\exp(s_{ti})}{\sum_j \exp(s_{tj})}, \quad s_{ti} = \frac{q_t^\top k_i}{\sqrt{d_k}} + M_{ti},$$

so $a_{ti} \geq 0$ and $\sum_i a_{ti} = 1$. Throughout the remainder of the paper, we consider the following causal mask: $M_{ti} = 0$ for $i \leq t$ and $-\infty$ otherwise. Further, we consider mostly a single decode-step attention, which corresponds to the current row of attention $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$ as introduced above, i.e.,

$$o_t = \sum_{i=1}^t a_{ti} v_i \mathbf{W}_O. \quad (3)$$

Remark 1. *Modern LLMs adopt different variants of attention mechanisms (e.g., group query attention [35]). Our method, described later, naturally extends to them (see Section V), but for now we will report everything as defined above.*

LLM inference has two phases. In the *prefill* phase, a prompt of T tokens is processed in parallel and for each layer ℓ , the keys $\mathbf{K}^{(\ell)} \in \mathbb{R}^{T \times d_k}$ and values $\mathbf{V}^{(\ell)} \in \mathbb{R}^{T \times d_v}$ are stored in the cache. In the *decode* phase, each step $t = T+1, T+2, \dots$ processes only the new token, appends its key k_t and value v_t , and computes attention for q_t against all T cached keys and values. The KV cache at step t is

$$\mathcal{C}_t = \{(\mathbf{K}_{\leq t}^{(\ell)}, \mathbf{V}_{\leq t}^{(\ell)}) \mid \ell \in [L]\},$$

where $\mathbf{K}_{\leq t}^{(\ell)} \in \mathbb{R}^{t \times d_k}$ has rows $k_1, \dots, k_t$ and its columns are called channels; analogously for $\mathbf{V}_{\leq t}^{(\ell)}$.

C. Quantization of the KV cache

Definition 2 (Uniform Scalar Quantization (following [36])). A b -bit uniform scalar quantizer with scale $\Delta > 0$ and zero-point $z \in \mathbb{R}$ maps a scalar $x \in \mathbb{R}$ to a quantized integer and back via

$$\text{qtz}(x) = \text{clamp}\left(\left\lfloor \frac{x - z}{\Delta} + \frac{1}{2} \right\rfloor, 0, 2^b - 1\right),$$

$$\text{dqtz}(c) = \Delta \cdot c + z,$$

where $\lfloor \cdot \rfloor$ denotes the floor function and $\text{clamp}(x, a, b) = \min(\max(x, a), b)$ clips the range to $[a, b]$. The reconstruction $\hat{x} = \text{dqtz}(\text{qtz}(x))$ satisfies $|x - \hat{x}| \leq \Delta/2$ for values within the clipping range.

A quantized cache replaces each k_i, v_i by $\hat{k}_i = \text{dqtz}(\text{qtz}(k_i))$, $\hat{v}_i = \text{dqtz}(\text{qtz}(v_i))$, with error vectors $\delta k_i = \hat{k}_i - k_i$, $\delta v_i = \hat{v}_i - v_i$. The quantized output is $\hat{o}_t = \text{Attention}(\tilde{q}_t, \hat{\mathbf{K}}_{\leq t}, \hat{\mathbf{V}}_{\leq t})\mathbf{W}_O$. The pair (Δ, z) may be computed at different *granularities*: per-tensor (one pair per matrix); per-token/row-wise (one pair per k_i); per-channel/column-wise (one pair per dimension c , adapting to each channel’s range across tokens); or group quantization (one pair per contiguous group of G elements along a chosen axis). Transform-coding approaches such as KVTC [8] choose the quantizer to minimize the distortions $\mathbb{E}\|k_i - \hat{k}_i\|^2$ and $\mathbb{E}\|v_i - \hat{v}_i\|^2$ which only optimize over the reconstruction error of the keys and values, but not the reconstruction error of the attention output. Therefore, these are only proxies for the quantity of interest, a gap that [8] themselves acknowledge. The quantity of interest is the output distortion defined as follows.

Definition 3 (Attention output distortion). In line with the discussion in Section III-A, the expected squared-error distortion between the quantized attention output $\hat{o}_t$ and o_t , the attention output for token t , is defined as

$$D = \mathbb{E}\|o_t - \hat{o}_t\|_2^2,$$

averaged over the empirical distribution of keys and values.

Three effects drive the gap between these two distortion measures. First, the keys influence the attention output only through their alignment with the query. Second, a token with almost no attention weight a_i can be quantized coarsely for both its keys and values without increasing the output distortion. Third, value errors enter the attention output linearly and are reshaped by the output projection. Theorem 1 characterizes this relationship explicitly, and Section IV-B shows how the resulting decomposition subsumes several asymmetries that prior methods exploit only in isolation.

IV. ATTENTION-AWARE DISTORTION FORMULA FOR KV CACHE COMPRESSION

In this section, we decompose the difference between the outputs of the attention mechanism computed using the quantized and original KV caches. Theorem 1 decomposes the distortion, defined in Definition 3, into key- and value-induced contributions in which these factors appear as

explicit weights, yielding the distortion measure our bit allocation minimizes.

A. Derivation of the Distortion Formula

To make the distortion analytically tractable, we model the quantization noise under the standard additive white-noise hypothesis stated below and we assume the key quantization error to be bounded.

Assumption 1 (White-noise quantization model). The quantization errors $\delta k_{ic} = \hat{k}_{ic} - k_{ic}$, $\delta v_{ic} = \hat{v}_{ic} - v_{ic}$ (k_{ic} is the c -th element of the vector k_i , analogue for the values) are zero-mean, mutually independent across tokens i and channels c , independent between keys and values, and symmetrically distributed. In particular, all odd moments vanish, $\mathbb{E}[\delta k_{ic}^3] = \mathbb{E}[\delta v_{ic}^3] = 0$.

Assumption 2 (Bounded key quantization error). There exists a constant $\kappa \geq 1$ such that, almost surely,

$$|\delta k_{ic}| \leq \kappa \sigma_K \quad \text{for all tokens } i \text{ and channels } c.$$

Remark 2. The additive white-noise model of quantization (Assumption 1) is the standard idealization underlying rate-distortion analyses of quantization [37]–[39], even though it is known to be exact only in the high-rate limit, we demonstrate in our experiments that the distortion metric remains predictive at 2 – 4 bits. We adopt it in this paper, as for the uniform symmetric quantizer used in Section V, the zero-mean and symmetry properties are in exact analogy to the transform coding literature. Key and value independence follow similarly, since they arise from distinct projections. Cross-channel independence is justified along similar lines when the channels are decorrelated before quantization. In contrast, cross-token independence is an idealization: neighboring KV representations are correlated, so per-token errors are not strictly independent. Exploiting this dependence, however, would require a computationally intractable token varying bit allocation. For that reason, almost all quantization methods rely on an idealization along these lines. Accordingly, we adopt the standard independence approximation used in transform coding and most existing KV cache quantization methods.

Assumption 2 corresponds to boundedness of the token entries as the max-based token scaling (Section V) is non-overloading. Namely, the quantization step scales with the maximum entry and each key error is at most half a quantization step (Definition 2).

The following theorem shows that under this model, the distortion, defined in 3, decomposes into a key term and a value term, each factoring across tokens and channels.

Theorem 1 (Attention Output Distortion). Consider the single decode-step attention model of (3) and assume that its quantized keys $\hat{k}_i = k_i + \delta k_i$ and its quantized values $\hat{v}_i = v_i + \delta v_i$ satisfy the white-noise hypothesis (Assumption 1) and assume that δk_i is bounded (Assumption 2). Then the expected squared distortion of the attention output D separates into additive key and value contributions,

$$D = \mathbb{E}[\|o - \hat{o}\|^2] = D_K + D_V + R, \quad (4)$$

$$D_K = \sum_{i,c} a_i^2 \|v_i \mathbf{W}_O - o\|^2 q_c^2 \mathbb{E}[(\delta k_{ic})^2],$$

$$D_V = \sum_{i,c} a_i^2 \mathbb{E}[(\delta v_{ic})^2] \|\mathbf{W}_{O_c}\|_F^2,$$

each factoring across tokens i and channels c , dropping the token index t in o_t for readability. The remainder is higher order in σ_K and σ_V ,

$$|R| \leq C \|q\|^2 \|\mathbf{W}_O\|_F^2 \sigma_K^2 \sigma_V^2 + C (A_0^3 \|\mathbf{W}_O\|_F^2 \sigma_V^2 + D_o A_0^4),$$

where $A_0 = \kappa \|q\|_1 \sigma_K$, $D_o := \max_i \|v_i \mathbf{W}_O - o\|^2$ and C is absolute constant. R is negligible whenever the effective noise scales A_0 and $\|\mathbf{W}_O\|_F^2 \sigma_V^2$ are small.

Remark 3 (Remainder term). *The remainder measures the difference between the exact output distortion and its second-order approximation. The theoretical derivation guarantees that it vanishes asymptotically as the quantization noise decreases. In practice, we work in low-bit regimes, the experiments in Section VI suggest that the approximation even remains predictive in those regimes.*

We will continue with a proof sketch of Theorem 1, the full proof is given in Appendix A.

Proof Sketch Theorem 1. In short, we expand the output error to second order in the quantization noise, take expectations, keep the leading terms, and bound the rest.

First, subtracting the quantized output from the exact one cancels the unperturbed term and splits the error into three parts, cancels against the corresponding term in o :

$$o - \hat{o} = - \left(\underbrace{\sum_{i=1}^T \delta a_i v_i \mathbf{W}_O}_{=: E_K} + \underbrace{\sum_{i=1}^T a_i \delta v_i \mathbf{W}_O}_{=: E_V} + \underbrace{\sum_{i=1}^T \delta a_i \delta v_i \mathbf{W}_O}_{=: E_\times} \right),$$

where E_K collects the effect of the key error through the perturbed attention weights δa_i , E_V the effect of the value error, and $E_\times$ a second-order cross-term. A first- and second-order Taylor expansion of the softmax writes the weight perturbation as $\delta a_i = \delta a_i^{(1)} + \delta a_i^{(2)} + r_i$, with the remainder r_i controlled by Lemma 1.

Second, expanding $\mathbb{E}[\|E_K + E_V + E_\times\|^2]$ and using that the key and value errors are independent and zero-mean (Assumption 1), the key-value cross-terms $\mathbb{E}[E_K E_V^\top]$ and $\mathbb{E}[E_K E_\times^\top]$ vanish exactly, and the remaining cross-terms are higher order. The distortion reduces to $\mathbb{E}[\|E_K\|^2] + \mathbb{E}[\|E_V\|^2]$ plus a remainder.

Third, for the key term, recentering each value by the output o leaves E_K unchanged, since the weight perturbations sum to zero and channel independence then gives $\mathbb{E}[E_K E_\times^\top] = \sum_{i,c} a_i^2 \|v_i \mathbf{W}_O - o\|^2 q_c^2 \mathbb{E}[(\delta k_{ic})^2] = D_K$ up to higher-order terms.

Finally, the value term is exact. With $E_V = \sum_i a_i^2 (\delta v_i)^2 \mathbf{W}_O$ and deterministic a_i , token independence and zero mean leave only the diagonal, giving $D_V = \sum_{i,c} a_i^2 \mathbb{E}[(\delta v_{ic})^2] \|\mathbf{W}_{O_c}\|_F^2$.

Collecting the two leading terms and the bounded higher-order contributions yields $D = D_K + D_V + R$, with R controlled through $A_0 = \kappa \|q\|_1 \sigma_K$.

B. Interpretation of the Distortion Measure

a) *Key distortion D_K* : Each channel c of the key token is weighted by (a) the squared attention weight a_i^2 , reflecting how much the token participates in the output; (b) the residual $\|v_i \mathbf{W}_O - o\|^2$, measuring how much switching attention to this token would change the output; and (c) the squared query component q_c^2 , capturing which key channels the current query actually reads. Therefore, compressing channels for which q_c^2 is small or deleting tokens for which a_i^2 or $\|v_i \mathbf{W}_O - o\|^2$ is small does not change the distortion much.

b) *Value distortion D_V* : Each channel c of value token i is weighted by a_i^2 , so only tokens that receive significant attention contribute to value distortion. The output projection $\mathbf{W}_{O_c}$ further scales the contribution of each value channel.

A central observation is that the distortion in (4) factorizes: the token-dependent weights (a_i^2 , $\|v_i \mathbf{W}_O - o\|^2$) and the channel-dependent weights (q_c^2 , $\mathbf{W}_{O_c}^2$) appear in distinct factors. Under Assumption 1, $\mathbb{E}[(\delta k_{ic})^2]$ only depends on the bit allocation b_c for channel c but is the same across tokens. Therefore, the total distortion factors are

$$D_K = \left(\sum_i a_i^2 \|v_i \mathbf{W}_O - o\|^2 \right) \cdot \left(\sum_c q_c^2 \mathbb{E}[(\delta k_{ic})^2] \right), \quad (5)$$

and analogously for D_V . This separation has two consequences. First, it implies that we can focus on either channel-wise or token-wise compression and design a quantizer that allocates bits accordingly. Second, the keys and values can be quantized independently: D_K and D_V are separate terms, with no cross-term coupling key and value quantization errors. Existing KV cache compression methods can be classified by which of the factors above they target. Table I summarizes this classification. A detailed explanation that identifies the methods addressing each factor and describes our method's position relative to them can be found in Appendix C. Our method synthesizes the channel-level factors of the decomposition; the token-wise allocation is out of scope here.

V. AATC: ATTENTION-AWARE BIT ALLOCATION VIA TRANSFORM CODING

The channel-wise allocation determines how many bits to assign to each hidden dimension of the key and value cache. As in classical transform coding, we compute it via reverse waterfilling on an empirical distribution obtained in advance from a calibration dataset. Such a set can, however, only capture the varying importance of *channels*, not of individual *tokens*. The allocation, therefore, operates per token, distributing bits across channels. The procedure consists of two steps: (i) a linear transform that decorrelates and importance-orders the dimensions, and (ii) an optimization that assigns bits to each transformed dimension so as to minimize the attention-aware distortion under a global budget.

TABLE I

DISTORTION FACTORS IN $D = D_K + D_V$ AND THE DESIGN AXES THEY MOTIVATE. EACH ROW CORRESPONDS TO ONE FACTOR IN THE PER-TOKEN, PER-CHANNEL DECOMPOSITION OF ATTENTION-OUTPUT ERROR UNDER KV QUANTIZATION. METHODS ARE LISTED BY THE FACTOR THEY PRIMARILY TARGET. HERE, VQ=VECTOR QUANTIZATION, MP = MIXED PRECISION AND TC = TRANSFORM CODING.

Factor	Design axis	Mechanism	Representative methods
a_i^2 (token)	token eviction	drop tokens	H2O [9], SnapKV [22], StreamingLLM [10], VATP [23], CriticalKV [24], Expected Attention [26]
$\ v_i \mathbf{W}_O - o\ ^2$ (token)	output-relevance	drop tokens	CAOTE [25](pre- $\mathbf{W}_O$)
q_c^2, C_Q (K channel)	query-aware key precision	VQ/MP/rotation/ TC+bit-alloc.	A ² ATS [12], MixKVQ [21], OSCAR [19], SQuat [20], Ours (Section V)
$\mathbf{W}_{O_c}$ (V channel)	output-projection- aware value	TC+bit-alloc	(implicit in KVQuant [6]), Ours (Section V)
$\mathbb{E}[(\delta k_{ic})^2],$ $\mathbb{E}[(\delta v_{ic})^2]$	per-channel sensitivity	scaling/codebook/ SVD/MP/ TC+bit-alloc	KIVI [5], KVQuant [6], KVTC [8], PALU [7], RateQuant [11], Ours (Section V)

A. Step 1: Whitening-Based Feature Decorrelation

KV caches are especially amenable to transform coding, since key and value vectors are strongly correlated across dimensions and their effective rank is typically below d_k [7], [8]. We achieve decorrelation by computing a whitening-based SVD of the weight matrices $\mathbf{W}_K$ and $\mathbf{W}_V$, which approximately satisfy the independence assumptions required for classical transform coding. Given calibration activations $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$ and a projection matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{model}} \times d}$ (either $\mathbf{W}_K$ or $\mathbf{W}_V$), we proceed as follows to compute a transformed representation $\mathbf{H}$ (denoted by $\mathbf{H}^K$ or $\mathbf{H}^V$, respectively). Define the empirical input covariance $\Sigma_X = \frac{1}{T} \mathbf{X}^\top \mathbf{X}$. Let $\Sigma_X = \mathbf{L} \mathbf{L}^\top$ be its Cholesky decomposition. The *whitened weight matrix* is $\widetilde{\mathbf{W}} = \mathbf{L}^\top \mathbf{W}$, which absorbs the input statistics into the projection. Let $\widetilde{\mathbf{W}} = \mathbf{U} \mathbf{S} \mathbf{P}^\top$ be its reduced SVD. We then factorize the original projection as

$$\mathbf{W} = \mathbf{L}^{-\top} \mathbf{U} \mathbf{S} \mathbf{P}^\top = \mathbf{A} \mathbf{B}, \quad (6)$$

with $\mathbf{A} = \mathbf{L}^{-\top} \mathbf{U} \sqrt{\mathbf{S}}, \quad \mathbf{B} = \sqrt{\mathbf{S}} \mathbf{P}^\top.$

At inference time, instead of caching $\mathbf{K} = \mathbf{X} \mathbf{W}_K \in \mathbb{R}^{T \times d_k}$, we cache the transformed representation

$$\mathbf{H}^K = \mathbf{X} \mathbf{A}^K \in \mathbb{R}^{T \times d_k}, \quad (7)$$

and recover the original keys via $\mathbf{K} = \mathbf{H}^K \mathbf{B}^K$. Same for the values, we have $\mathbf{H}^V = \mathbf{X} \mathbf{A}^V$ and therefore $\mathbf{V} = \mathbf{H}^V \mathbf{B}^V$. In the transformed space, the dimensions of $\mathbf{H}^K$ and $\mathbf{H}^V$ are decorrelated and ordered by the singular values $\mathbf{S} = \mathbf{A}^\top \Sigma_X \mathbf{A} = \mathbf{H}^\top \mathbf{H}$, which capture the combined effect of input variance and projection magnitude. High-index dimensions correspond to small singular values and contribute little to the output, making them natural candidates for low-precision or zero-bit allocation.

Remark 4. In practice, one often considers a so-called *multi-head attention*, where each $\mathbf{W}$ is split into H sub-matrices ($\mathbf{W}_1, \dots, \mathbf{W}_H \in \mathbb{R}^{d_{\text{in}} \times d_k/H}$) referred to as *heads* and computes the attention individually for each head

before eventually combining it again. For the transform coding step in such a setup, the heads get combined into G head-groups and therefore we have $\mathbf{B}_1, \dots, \mathbf{B}_G$. Further, $\mathbf{B}_i^K \in \mathbb{R}^{d_k/G}$, same for the values we have $\mathbf{B}_i^V \in \mathbb{R}^{d_v/G}$. $\mathbf{A}$, on the other hand, remains global.

The above transform is the same as the one used in [7] in the context of dimension reduction and its decorrelation step coincides with the PCA of $\mathbf{K}$, hence with the transform of a per-layer KVTC [8], for details see Appendix D. The equivalence concerns the transform only: KVTC operates on a global, cross-layer cache and allocates based on reconstruction error, whereas we operate per layer under a global bit budget and allocate based on the output-aware distortion of Theorem 1.

B. Step 2: Attention-Aware Bit Allocation via Waterfilling

With the transform in hand, we formulate the channel-wise bit allocation as a constrained optimization problem. A detailed derivation of it can be found in Appendix B.

a) *The waterfilling problem for the keys and values:*

The channel-wise allocation for the keys is the solution to

$$\min_{\{b_{\ell,c}\}} \sum_{\ell=1}^L \sum_{c=1}^{d_k} w_{\ell,c}^{(K)} (\sigma_{\ell,c}^{(K)})^2 2^{-2b_{\ell,c}} \quad \text{s.t.} \quad \sum_{\ell=1}^L \sum_{c=1}^{d_k} b_{\ell,c} \leq \mathcal{B}_K, \quad (8)$$

$$\text{with } w_{\ell,c}^{(K)} = \frac{1}{T} \sum_{i=1}^T ((\mathbf{B}_{\ell,c}^K)^\top q_{(i,\ell,c)})^2$$

a per-channel weight with i being the tokens from the calibration set, T the size of the calibration set and $b_{\ell,c} \in \{0, 1, \dots, b_{\max}\}$ where $\mathcal{B}_K$ is the total bit budget for the keys. The continuous relaxation of (8) admits a weighted variant of the classical closed-form *reverse waterfilling* solution [3, Ch. 10]. A channel receives $b_{\ell,c}^* = \frac{1}{2} \log_2(w_{\ell,c}^{(K)} (\sigma_{\ell,c}^{(K)})^2 / \lambda)$ bits when $w_{\ell,c}^{(K)} (\sigma_{\ell,c}^{(K)})^2 > \lambda$ and zero bits otherwise, with the water level $\lambda > 0$ chosen so that $\sum_{\ell} \sum_c b_{\ell,c}^* = \mathcal{B}_K$. Channels below the water level are

discarded entirely. The waterfilling problem for the values reads as

$$\min_{\{b_{\ell,c}\}} \sum_{\ell=1}^L \sum_{c=1}^{d_v} w_{\ell,c}^{(V)} (\sigma_{\ell,c}^{(V)})^2 2^{-2b_{\ell,c}} \text{ s.t. } \sum_{\ell=1}^L \sum_{c=1}^{d_v} b_{\ell,c} \leq \mathcal{B}_V,$$

with $w_{\ell,c}^{(V)} = \|(\mathbf{B}^V \mathbf{W}_O)_{\ell,c}^\top\|_F^2$ and $b_{\ell,c} \in \{0, 1, \dots, b_{\max}\}$.

b) *The right scaling:* After fixing the bits per channel, one needs to think about how to choose the scale to apply uniform scalar quantization (Definition 2) to the fixed number of bits. This we do in the following way: we first calculate a channel scale ($s_{\ell,c}$) on the calibration data by calculating the 99th percentile of $|h_{\ell,c}|$ across all calibration tokens for every channel existing for keys and for values. During inference time, we compute the token-wise scaling for every token in every layer as $\alpha_{\ell,t} = \max_c(|h_{\ell,c,t}/s_{\ell,c}|)$ and quantize

$$\text{qtz}(h) = \text{clamp}\left(\left\lfloor \frac{h}{\Delta_{\ell,c,t}} + \frac{1}{2} \right\rfloor, -2^{b_{\ell,c}-1}, 2^{b_{\ell,c}-1} - 1\right),$$

$$\text{dqtz}(c) = \Delta_{\ell,c,t} \text{qtz}(h) \text{ with } \Delta_{\ell,c,t} := \frac{2\alpha_{\ell,t} s_{\ell,c}}{2^{b_{\ell,c}}}.$$

In practice we actually compute the token-wise scaling per head-group g (defined in Remark 4), so we obtain $\alpha_{\ell,g,t} = \max_{c \in g}(|h_{\ell,c,t}/s_{\ell,c}|)$

c) *Separate budgets for keys and values.:* Keys and values play asymmetric roles in attention, this is why we solve separate allocation problems for $\mathbf{W}_K$ and $\mathbf{W}_V$, each with its own budget $\mathcal{B}_K$ and $\mathcal{B}_V$.

Remark 5 (Zero-bit dimensions and rank reduction). *The dynamic program naturally assigns zero bits to trailing dimensions whose weight w_c is too small to justify even one bit, which is equivalent to rank reduction. The effective cache dimensionality is reduced from d_k to the number of dimensions receiving at least one bit. Unlike fixed-rank SVD truncation like PALU [7], the cutoff adapts to the attention-aware weight profile and the available budget. In contrast to our method, PALU does not optimize bit allocation. In the experiments, we also see favorable performance for our method compared to PALU (see Section VI).*

To summarize, our method operates in two stages: an offline calibration stage that determines the channel-wise bit allocation, and an online inference stage that applies quantization during inference. We summarize the full procedure of our method, called attention-aware transform coding (AATC), in Algorithm 1.

VI. EXPERIMENTS

To show the performance of our method we evaluate it on LongBench [40], RULER [41], GSM8K [42], MATH-500 [43] and MMLU-Pro [44] (math and computer science subtasks) to validate its qualitative performance against existing methods (KIVI, PALU and KVQuant) on Llama-3.1-8B-Instruct [45] and Qwen-2.5-7B-Instruct [46]. For KIVI and KVQuant, we reimplemented their method and, for fairness, added a recent-token window of size 128, which gets flushed in blocks of 32 tokens. We also kept the first 4 tokens in full

Algorithm 1 Attention-Aware Transform Coding (AATC) for KV cache Compression

Require: Model weights $\{\mathbf{W}_K^{(\ell)}, \mathbf{W}_V^{(\ell)}, \mathbf{W}_O^{(\ell)}\}$; calibration data $\mathbf{X}$; bit budget $\mathcal{B}$; min bits $b_{\min}$

Ensure: Compressed KV cache with attention-aware bit allocation

— **Calibration (offline, once per model)** —

- 1: **for** each layer ℓ **do**
- 2: Compute $\Sigma_X^{(\ell)}$, Cholesky $\mathbf{L}^{(\ell)}$
- 3: SVD of full $\mathbf{W}_K^{(\ell)}$: obtain $\mathbf{A}_\ell^K$ and $\mathbf{B}_\ell^K$
- 4: Run forward pass; record latent variance $(\sigma_{\ell,c}^{(K)})^2$
- for** each $c \in \{1, \dots, d_k\}$
- 5: Compute $w_{\ell,c}^{(K)} \leftarrow \frac{1}{T} \sum_i (\mathbf{B}_\ell^K[c, :] \cdot q_i)^2$
- 6: Compute $\mathbf{B}_\ell^V, \sigma_{\ell,c}^{(V)}, w_{\ell,c}^{(V)} \leftarrow \|(\mathbf{B}^V \mathbf{W}_O)_{\ell,c}^\top\|_F^2$
- 7: **end for**
- 8: Allocate bits globally via reverse waterfilling over all (ℓ, c)

$$\{b_c^{(K,\ell)}\} \leftarrow \arg \min_{\sum_{\ell,c} b_c^{(K)} \leq \mathcal{B}_K} \sum_{\ell,c} \left(w_{\ell,c}^{(K)} \cdot (\sigma_{\ell,c}^{(K)})^2 2^{-2b_{\ell,c}} \right)$$

$$\{b_c^{(V,\ell)}\} \leftarrow \arg \min_{\sum_{\ell,c} b_c^{(V)} \leq \mathcal{B}_V} \sum_{\ell,c} \left(w_{\ell,c}^{(V)} \cdot (\sigma_{\ell,c}^{(V)})^2 2^{-2b_{\ell,c}} \right)$$

— **Inference (online, per decode step t)** —

- 9: **for** each new token at step t **do**
 - 10: **for** each layer ℓ **do**
 - 11: Project: $h_{\ell,t}^K \leftarrow x_t^{(\ell)} \mathbf{A}_\ell^K, h_{\ell,t}^V \leftarrow x_t^{(\ell)} \mathbf{A}_\ell^V$
 - 12: Append $\text{qtz}(h_{\ell,t}^K)$ and $\text{qtz}(h_{\ell,t}^V)$ to cache
 - 13: Reconstruct $K_i^{(\ell)} \leftarrow \hat{h}_{\ell,i}^K \mathbf{B}_\ell^K, V_i^{(\ell)} \leftarrow \hat{h}_{\ell,i}^V \mathbf{B}_\ell^V$
 - 14: Compute $\alpha_{t,i} \leftarrow \text{softmax}(q_t K_i^\top / \sqrt{d})$
 - 15: Output $o_t^{(\ell)} \leftarrow \sum_{i \leq t} \alpha_{t,i} V_i^{(\ell)}$
 - 16: **end for**
 - 17: **end for**
-

precision. AATC is implemented as described in Algorithm 1. We further keep the first s tokens and the most recent w tokens in full precision, following standard practice [8], [10]. We use $s = 4$ and $w = 128$. Once the recent-token window is full, we quantize the oldest 16 tokens and refill until the window again holds w tokens. For PALU, we applied a transform with 30% dimension reduction and afterwards quantized to 4 bits. Further, we used $s = 4$ and $w = 128$ for the non-zero dimensions. We further evaluate AATC var-only, which uses the same per-layer transform and reverse-waterfilling allocation but drops the attention-aware weights q_c^2 and $W_{O,c}$, allocating on the quantization-error variance alone. KVTC [8] is the closest method to ours: it likewise decorrelates the cache and allocates bits by transform coding. It differs on two axes: it uses a single global, cross-layer transform (we use per-layer transforms under a global budget, the natural granularity given that each layer maintains its own KV cache) and it allocates on reconstruction error (we allocate on the output-aware distortion of Theorem 1, via q_c^2 and $W_{O,c}$). In the per-layer setting, the two coincide: the transforms

match (Appendix D) and the var-only’s variance-based allocation is exactly the reconstruction error criterion, so the var-only variant above can be interpreted as a per-layer analog of the KVTC algorithm. For more information on the methods, see Appendix E. Llama has 8 KV-heads, 32 Query heads and 32 layers. Qwen has 4 KV-heads, 28 Query heads and 28 layers. Therefore, both methods use grouped query attention. Our method we use with $G = 2$ head-groups for Llama and $G = 1$ head-group for Qwen. To generate the transform AB for the whitening-based feature decorrelation, we use wikitext2 [47]. For the rate-distortion calibration, we use a balanced mixture of FineWeb [48] (sample-10BT, web text) and OpenR1-Math-220k [49] (mathematical reasoning traces). We draw 256 calibration sequences of length 2048 tokens, split equally between FineWeb and OpenR1-Math-220k (128 sequences each), and shuffle them together before calibration. The mixture is intended to provide coverage over both general-domain and structured reasoning text distributions, following the approach of KVTC [8].

A. Main Results

Table II reports the main comparison; we mark in bold the best compressed method. The FP16 model is a reference and is not bolded. For each benchmark we take σ to be the standard error of its score, which is the standard deviation of the per-example scores divided by $\sqrt{N}$ over the N evaluated samples. We call a method statistically indistinguishable from FP16 when its score lies within 2σ of the FP16 score on that benchmark.

Across all 18 cells, AATC is never statistically distinguishable from the uncompressed FP16 model. At $5.82\times$ compression it is lossless to within evaluation noise on every benchmark and both backbones. This operating point reduces the memory budget of Llama from 1.07 GB for the FP16 baseline to 184MB. On Qwen, it is moreover the unique best compressed method on long-context retrieval (RULER-32k) and on multiple-choice reasoning (MMLU-Pro math), exceeding the strongest quantization baseline by 7.2 and 16.8 points. The two backbones respond differently to compression. On Llama, where even naive quantization (KIVI) stays near-lossless, AATC and AATC var-only are statistically tied throughout. On Qwen, which is markedly harder to compress (KIVI falls to 0.351 on LongBench and below 0.28 on RULER), the additional adaptivity of the full method gives consistent gains over AATC var-only (RULER-32k +2.5, MMLU-m +3.6). PALU is competitive only at short contexts, where it is best at RULER-4k on Qwen, but it attains the weakest LongBench average (0.483), a behavior we attribute to its low-rank parameterization. For a detailed explanation, see Appendix G.

We report further analyses in the appendix: a per-subtask LongBench breakdown (in Appendix G), an average bit sweep from 4-bit to 1-bit (Appendix H), detailed results at the aggressive 2-bit setting (Appendix H), an ablation of the allocation and calibration choices (Appendix I), full implementation details for all methods (Appendix E) and details about the benchmarks (Appendix F).

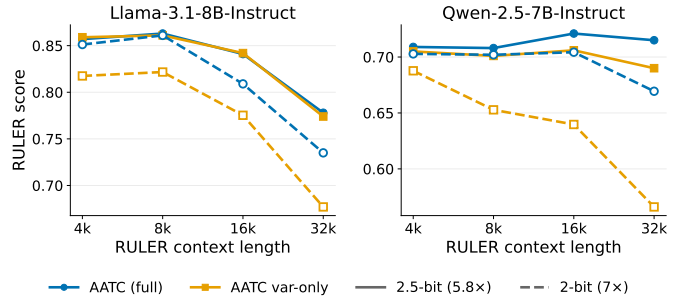


Fig. 1. RULER score versus context length for AATC and AATC var-only at 2.5-bit (solid) and 2-bit (dashed), on Llama-3.1-8B-Instruct and Qwen-2.5-7B-Instruct.

B. Benefits of Attention-Awareness

At the $5.82\times$ operating point of Table II, AATC var-only is statistically indistinguishable from full AATC for Llama, which raises the question of whether including the q_c^2 and W_O terms in the bit-allocation is necessary for all models. Figure 1 answers this by evaluating both variants at a more aggressive $\approx 7\times$ compression, corresponding to a 2-bit budget. At 2.5-bit (solid), the two curves are indistinguishable at every context length for Llama. For Qwen we already see a small advantage of AATC. At 2-bit (dashed) the full method retains near-baseline RULER accuracy across lengths for both models, whereas AATC var-only degrades sharply as the context grows. The more we compress, the better we can see the effect of the attention-aware allocation. We see, especially under aggressive compression and in long-context regimes, which is what we care about.

VII. LIMITATIONS AND FUTURE WORK

On the theoretical side, our analysis assumes that tokens are independent, which may not always hold in practice, since natural language can exhibit non-trivial correlation structures. While this assumption is common in related works and does not seem to have a strong negative impact on the encoding quality, we still think characterizing the dependence, or deriving guarantees under weaker assumptions, is an interesting direction for future theoretical work. On the empirical side, our implementation is not system-level optimized, in particular, it lacks a dedicated CUDA implementation, which would be required before the method could be used in production. Our evaluation also covers only two model families (Llama and Qwen), while these differ in scale and attention configuration, testing on a wider range of models is left for future work. Finally, it would be interesting to study how our method interacts with token-eviction methods [9], [10], [22] or how it can be extended to a global cache as it is deployed in [8].

TABLE II

COMP. IS THE KV CACHE COMPRESSION RATIO RELATIVE TO THE FP16 BASELINE. LONGBENCH IS THE TASK-AVERAGED SCORE OVER SEVEN ENGLISH SUBSETS (PER-SUBTASK BREAKDOWN IN TABLE III). HERE σ IS THE STANDARD ERROR OF EACH SCORE, THE STANDARD DEVIATION OF THE PER-EXAMPLE SCORES DIVIDED BY $\sqrt{N}$ OVER THE N EVALUATED SAMPLES. A METHOD IS CALLED STATISTICALLY INDISTINGUISHABLE FROM FP16 WHEN ITS SCORE LIES WITHIN 2σ OF THE FP16 SCORE.

Method	Comp.	LongBench avg.	RULER				Reasoning			
			4k	8k	16k	32k	GSM8K	MLLU-m	MLLU-cs	MATH-500
Llama-3.1-8B-Instruct										
Baseline	1.00	0.570	0.859	0.856	0.828	0.775	0.789	0.438	0.456	0.454
KIVI	5.00	0.564	0.846	0.840	0.810	0.763	0.730	0.390	0.420	0.452
PALU	5.39	0.550	0.851	0.849	0.83	0.769	0.617	0.274	0.320	0.314
KVQuant	5.91	0.549	0.715	0.729	0.676	0.654	0.754	0.376	0.402	0.392
AATC var	5.82	0.566	0.859	0.861	0.842	0.774	0.780	0.402	0.449	0.490
AATC (ours)	5.82	0.558	0.857	0.863	0.841	0.778	0.773	0.416	0.442	0.488
Qwen-2.5-7B-Instruct										
Baseline	1.00	0.574	0.720	0.716	0.718	0.720	0.818	0.708	0.567	0.680
KIVI	5.00	0.351	0.279	0.224	0.277	0.233	0.165	0.166	0.222	0.680
PALU	5.39	0.483	0.725	0.690	0.682	0.625	0.672	0.542	0.454	0.492
KVQuant	5.91	0.558	0.694	0.674	0.667	0.643	0.801	0.478	0.490	0.680
AATC var	5.82	0.558	0.705	0.701	0.706	0.690	0.824	0.674	0.551	0.668
AATC (ours)	5.82	0.567	0.709	0.708	0.721	0.715	0.822	0.710	0.573	0.680

APPENDIX

A. Proof of Theorem 1

Proof of Theorem 1. We proceed in four steps: (1) express the output error in terms of the perturbations δa_i , δv_i ; (2) show that the cross-terms vanish or are of higher order in expectation; (3) compute the key distortion term; (4) compute the value distortion term. Throughout, C denotes an absolute constant depending only on q , $\mathbf{W}_O$, and the attention weights $\{a_i\}$. Its value may change from line to line. We write $\sigma_K^2 := \max_{i,c} \mathbb{E}[(\delta k_{ic})^2]$ and $\sigma_V^2 := \max_{i,c} \mathbb{E}[(\delta v_{ic})^2]$.

Step 1: Decomposition of the output error. Let $\hat{a}_i$ denote the attention weights computed from the quantized keys and define the attention-weight perturbation $\delta a_i := \hat{a}_i - a_i$, so that $\hat{a}_i = a_i + \delta a_i$. The unperturbed and perturbed outputs are $o = \sum_{i=1}^T a_i v_i \mathbf{W}_O$ and $\hat{o} = \sum_{i=1}^T \hat{a}_i \hat{v}_i \mathbf{W}_O = \sum_{i=1}^T (a_i + \delta a_i)(v_i + \delta v_i) \mathbf{W}_O$. Multiplying out the product $(a_i + \delta a_i)(v_i + \delta v_i) = a_i v_i + a_i \delta v_i + \delta a_i v_i + \delta a_i \delta v_i$ and subtracting, the unperturbed term $a_i v_i$ cancels against the corresponding term in o :

$$\begin{aligned}
o - \hat{o} &= \sum_{i=1}^T a_i v_i \mathbf{W}_O \\
&\quad - \sum_{i=1}^T [a_i v_i + a_i \delta v_i + \delta a_i v_i + \delta a_i \delta v_i] \mathbf{W}_O \\
&= - \left(\underbrace{\sum_{i=1}^T \delta a_i v_i \mathbf{W}_O}_{=: E_K} + \underbrace{\sum_{i=1}^T a_i \delta v_i \mathbf{W}_O}_{=: E_V} + \underbrace{\sum_{i=1}^T \delta a_i \delta v_i \mathbf{W}_O}_{=: E_\times} \right),
\end{aligned}$$

and consequently

$$\mathbb{E}[\|o - \hat{o}\|^2] = \mathbb{E}[\|E_K + E_V + E_\times\|^2]. \quad (9)$$

The term E_K captures the effect of key quantization (through the perturbed attention weights), E_V captures

the effect of value quantization and $E_\times$ is a second-order cross-term involving both perturbations.

Softmax expansion. Both the key distortion (computed in Step 3 below) and the cross-terms (computed in Step 2 below) require the response of the attention weights to the key perturbation. Because the attention logits are given by $q^\top k_i$, the quantized keys perturb them by

$$\begin{aligned}
q^\top \hat{k}_i &= q^\top (k_i + \delta k_i) = q^\top k_i + \alpha_i, \\
\text{where } \alpha_i &:= q^\top \delta k_i = \sum_c q_c \delta k_{ic},
\end{aligned}$$

so α_i is the scalar perturbation to the i -th logit induced by the key error. Under Assumption 1 the δk_{ic} are independent and zero-mean, hence so are the α_i , so, in particular, $\mathbb{E}[\alpha_i \alpha_\ell] = 0$ for $i \neq \ell$. Furthermore, expanding the square of the sum and using channel independence, $\mathbb{E}[\delta k_{ic} \delta k_{ic'}] = \mathbb{1}_{c=c'} \mathbb{E}[(\delta k_{ic})^2]$, one obtains that,

$$\mathbb{E}[\alpha_i^2] = \mathbb{E}\left[\left(\sum_c q_c \delta k_{ic}\right)^2\right] = \sum_c q_c^2 \mathbb{E}[(\delta k_{ic})^2] \leq \|q\|^2 \sigma_K^2. \quad (10)$$

The last step uses that $\mathbb{E}[(\delta k_{ic})^2] \leq \sigma_K^2$ and $\sum_c q_c^2 = \|q\|^2$. Moreover, as a linear combination of independent symmetric variables α_i is itself symmetric and all of its odd moments vanish, in particular $\mathbb{E}[\alpha_i] = \mathbb{E}[\alpha_i^3] = 0$. Under Assumption 2, the logit perturbations are moreover bounded, which means $|\alpha_i| \leq \sum_c |q_c| |\delta k_{ic}| \leq \kappa \|q\|_1 \sigma_K := A_0$ almost surely. The perturbations will be centered at the attention-weighted mean $\bar{\alpha} := \sum_{\ell=1}^T a_\ell \alpha_\ell$. The weights a_ℓ being the unperturbed attention weights leads to $a_\ell \geq 0$ and $\sum_\ell a_\ell = 1$. Since $\bar{\alpha}$ is a convex combination of the α_ℓ , also $|\bar{\alpha}| \leq A_0$ and $|\alpha_i - \bar{\alpha}| \leq 2A_0$ almost surely. Writing $s_\ell := q^\top k_\ell$ for the unperturbed logits, the attention weight is $\hat{a}_i = \text{softmax}(s + \alpha)_i$. Decompose the attention-weight

perturbation as

$$\delta a_i = \underbrace{a_i(\alpha_i - \bar{\alpha})}_{=: \delta a_i^{(1)}} + \underbrace{\frac{1}{2} a_i \left[(\alpha_i - \bar{\alpha})^2 - \sum_{\ell} a_{\ell} (\alpha_{\ell} - \bar{\alpha})^2 \right]}_{=: \delta a_i^{(2)}} + r_i \quad (11)$$

where $\delta a_i^{(1)}$ is linear, $\delta a_i^{(2)}$ quadratic in the entries of α and $r_i := \delta a_i - \delta a_i^{(1)} - \delta a_i^{(2)}$ is defined as the exact difference. By Remark 6, $\delta a_i^{(1)}$ and $\delta a_i^{(2)}$ are precisely the first- and second-order Taylor terms of the softmax response $\theta \mapsto \text{softmax}(s + \theta \alpha)_i$ at $\theta = 0$, so r_i is its exact second-order Taylor remainder. It is bounded in Lemma 1 below. Because the $\hat{a}_i$ and a_i are both probability distributions ($\sum_i \hat{a}_i = \sum_i a_i = 1$), their difference satisfies

$$\sum_i \delta a_i = \sum_i (\hat{a}_i - a_i) = 0. \quad (12)$$

We use this expansion throughout Steps 2–4.

Step 2: Controlling the cross-terms in expectation. Expanding the squared norm in (9) yields,

$$\|E_K + E_V + E_{\times}\|^2 = \|E_K\|^2 + \|E_V\|^2 + \|E_{\times}\|^2 + 2E_K E_V^{\top} + 2E_K E_{\times}^{\top} + 2E_V E_{\times}^{\top}. \quad (13)$$

We will now show that $\mathbb{E}[E_K E_V^{\top}]$ and $\mathbb{E}[E_K E_{\times}^{\top}]$ vanish exactly, while $\mathbb{E}[E_V E_{\times}^{\top}]$ and $\mathbb{E}[\|E_{\times}\|^2]$ are of higher order and collected into the remainder. The leading terms $\mathbb{E}[\|E_K\|^2]$ and $\mathbb{E}[\|E_V\|^2]$ are evaluated in Steps 3 and 4.

(a) *The term $\mathbb{E}[E_K E_V^{\top}]$ vanishes exactly.* Writing both factors as sums gives,

$$\begin{aligned} \mathbb{E}[E_K E_V^{\top}] &= \sum_{i=1}^T \mathbb{E} \left[\sum_{\ell=1}^T \delta a_i a_{\ell} (v_i \mathbf{W}_O) (\delta v_{\ell} \mathbf{W}_O)^{\top} \right] \\ &= \sum_{i=1}^T \sum_{\ell=1}^T (v_i \mathbf{W}_O) \mathbb{E}_{\delta k} [\delta a_i] a_{\ell} \mathbf{W}_O^{\top} \mathbb{E}_{\delta v} [\delta v_{\ell}^{\top}] = 0. \end{aligned}$$

As the perturbation δa_i does not depend on the value errors δv_{ℓ} , the independence of key and value errors (Assumption 1) yields that δa_i and δv_{ℓ} are also independent and the last row follows, since $\mathbb{E}[\delta v_{\ell}] = 0$.

(b) *The term $\mathbb{E}[E_K E_{\times}^{\top}]$ vanishes exactly.* Analogously to (a), $\delta a_i \delta a_{\ell}$ and δv_{ℓ} are independent. Thus,

$$\begin{aligned} \mathbb{E}[E_K E_{\times}^{\top}] &= \sum_{i=1}^T \sum_{\ell=1}^T \mathbb{E}[\delta a_i \delta a_{\ell} (v_i \mathbf{W}_O) (\delta v_{\ell} \mathbf{W}_O)^{\top}] \\ &= \sum_{i=1}^T \sum_{\ell=1}^T \mathbb{E}_{\delta k} [\delta a_i \delta a_{\ell}] (v_i \mathbf{W}_O) \mathbf{W}_O^{\top} \mathbb{E}_{\delta v} [\delta v_{\ell}^{\top}] = 0. \end{aligned}$$

(c) $\mathbb{E}[E_V E_{\times}^{\top}]$ is $\mathcal{O}((\|q\|^2 \sigma_K^2 + A_0^3) \|\mathbf{W}_O\|_F^2 \sigma_V^2)$. Expanding both factors,

$$\begin{aligned} \mathbb{E}[E_V E_{\times}^{\top}] &= \sum_{i=1}^T \sum_{\ell=1}^T \mathbb{E}[a_i \delta a_{\ell} (\delta v_i \mathbf{W}_O) (\delta v_{\ell} \mathbf{W}_O)^{\top}] \\ &= \sum_{i=1}^T \sum_{\ell=1}^T a_i \mathbb{E}_{\delta k} [\delta a_{\ell}] \mathbb{E}_{\delta v} [(\delta v_i \mathbf{W}_O) (\delta v_{\ell} \mathbf{W}_O)^{\top}]. \end{aligned}$$

The second equality holds, since δa_{ℓ} depends only on $\{\delta k_m\}$, which are independent of the value errors. By token independence and $\mathbb{E}[\delta v_i] = 0$, the value expectation vanishes for $i \neq \ell$, so only the diagonal $i = \ell$ survives therefore we get

$$\mathbb{E}[E_V E_{\times}^{\top}] = \sum_{i=1}^T a_i \mathbb{E}_{\delta k} [\delta a_i] \mathbb{E}_{\delta v} [\|\delta v_i \mathbf{W}_O\|^2].$$

It remains to evaluate $\mathbb{E}[\delta a_i]$. From the expansion (11), $\mathbb{E}[\delta a_i^{(1)}] = a_i \mathbb{E}[\alpha_i - \bar{\alpha}] = 0$ because the α_i are zero-mean (Assumption 1). The leading nonzero contribution is the second-order term, with the tail controlled by Lemma 1:

$$\begin{aligned} \mathbb{E}[\delta a_i] &= \mathbb{E}[\delta a_i^{(2)}] + \mathbb{E}[r_i], \\ \mathbb{E}[\delta a_i^{(2)}] &= \frac{1}{2} a_i \left(\mathbb{E}[(\alpha_i - \bar{\alpha})^2] - \sum_{\ell} a_{\ell} \mathbb{E}[(\alpha_{\ell} - \bar{\alpha})^2] \right), \end{aligned}$$

where $|\mathbb{E}[r_i]| \leq \mathbb{E}[r_i] \leq 7A_0^3$ by (26). The centered second moments are computed exactly once here and reused in (d) below. Using $\mathbb{E}[\alpha_{\ell} \bar{\alpha}] = a_{\ell} \mathbb{E}[\alpha_{\ell}^2]$ since, only the $m = \ell$ term of $\bar{\alpha}$ contributes, by token independence and zero mean, and $\mathbb{E}[\bar{\alpha}^2] = \sum_m a_m^2 \mathbb{E}[\alpha_m^2]$, we obtain

$$\mathbb{E}[(\alpha_{\ell} - \bar{\alpha})^2] = (1 - 2a_{\ell}) \mathbb{E}[\alpha_{\ell}^2] + \sum_m a_m^2 \mathbb{E}[\alpha_m^2] \leq 2\|q\|^2 \sigma_K^2. \quad (14)$$

Here, the final bound uses $\mathbb{E}[\alpha_m^2] \leq \|q\|^2 \sigma_K^2$ from (10), $1 - 2a_{\ell} \leq 1$, and $\sum_m a_m^2 \leq 1$. Hence $|\mathbb{E}[\delta a_i^{(2)}]| \leq C a_i \|q\|^2 \sigma_K^2$ and, in particular, $|\mathbb{E}[\delta a_i]| \leq C(a_i \|q\|^2 \sigma_K^2 + A_0^3)$. Combining the above bound with

$$\mathbb{E}[\|\delta v_i \mathbf{W}_O\|^2] = \sum_c \mathbb{E}[(\delta v_{ic})^2] \|(\mathbf{W}_O)_{c,:}\|^2 \leq \sigma_V^2 \|\mathbf{W}_O\|_F^2 \quad (15)$$

and the fact that the per-token bound carries its own factor a_i , so that the token sum collapses through $\sum_i a_i^2 \leq \sum_i a_i = 1$, we obtain the final bound

$$\begin{aligned} |\mathbb{E}[E_V E_{\times}^{\top}]| &\leq \sum_i a_i |\mathbb{E}[\delta a_i]| \sigma_V^2 \|\mathbf{W}_O\|_F^2 \\ &\leq C(\|q\|^2 \sigma_K^2 + A_0^3) \|\mathbf{W}_O\|_F^2 \sigma_V^2. \end{aligned} \quad (16)$$

(d) *The term $\mathbb{E}[\|E_{\times}\|^2]$ is $\mathcal{O}(\|q\|^2 \|\mathbf{W}_O\|_F^2 \sigma_K^2 \sigma_V^2)$.* Expanding the squared norm and using the independence of key and value noise,

$$\mathbb{E}[\|E_{\times}\|^2] = \sum_{i=1}^T \sum_{\ell=1}^T \mathbb{E}_{\delta k} [\delta a_i \delta a_{\ell}] \mathbb{E}_{\delta v} [(\delta v_i \mathbf{W}_O) (\delta v_{\ell} \mathbf{W}_O)^{\top}].$$

For $i \neq \ell$, the value factor and the key factor both vanish by token independence and zero mean, leaving the diagonal

$$\mathbb{E}[\|E_{\times}\|^2] = \sum_{i=1}^T \mathbb{E}[\delta a_i^2] \mathbb{E}[\|\delta v_i \mathbf{W}_O\|^2]. \quad (17)$$

The value factor bound is already bounded in (15). Therefore, we only need to bound the key factor. Its leading part $\delta a_i^{(1)} = a_i(\alpha_i - \bar{\alpha})$ given the bound in (14) yields

$$\mathbb{E}[(\delta a_i^{(1)})^2] = a_i^2 \mathbb{E}[(\alpha_i - \bar{\alpha})^2] \leq 2a_i^2 \|q\|^2 \sigma_K^2.$$

Squaring the full $\delta a_i = \delta a_i^{(1)} + \delta a_i^{(2)} + r_i$ gives

$$\mathbb{E}[\delta a_i^2] = \mathbb{E}[(\delta a_i^{(1)})^2] + \underbrace{2\mathbb{E}[\delta a_i^{(1)}(\delta a_i^{(2)} + r_i)]}_{=: c_i} + \mathbb{E}[(\delta a_i^{(2)} + r_i)^2].$$

The corrections c_i are controlled pathwise. We have that $|\alpha_\ell - \bar{\alpha}| \leq 2A_0$ and $\sum_\ell a_\ell(\alpha_\ell - \bar{\alpha})^2 \leq 4A_0^2$ almost surely, the latter because it is a convex combination of the terms $(\alpha_\ell - \bar{\alpha})^2 \leq 4A_0^2$. From those inequalities it follows directly, from (11), $|\delta a_i^{(1)}| \leq 2a_i A_0$ and $|\delta a_i^{(2)}| \leq 2a_i A_0^2$, while $|r_i| \leq 7A_0^3$ and $\sum_i \mathbb{E}[r_i^2] \leq 49A_0^6$ by Lemma 1. Summing over tokens with $\sum_i a_i = 1$ and $\sum_i a_i^2 \leq 1$,

$$\sum_i |c_i| \leq 8A_0^3 + 28A_0^4 + 8A_0^4 + 98A_0^6 \leq C A_0^3,$$

the final inequality holding for $A_0 \leq 1$ directly, and for $A_0 > 1$ because $\sum_i \mathbb{E}[\delta a_i^2] \leq \mathbb{E}[(\sum_i |\delta a_i|)^2] \leq 4$ and $\sum_i \mathbb{E}[(\delta a_i^{(1)})^2] \leq 2A_0^2$ give $\sum_i |c_i| \leq 4 + 2A_0^2 \leq 6A_0^3$ outright. Therefore, the bound on the attention distortion terms is $\sum_i \mathbb{E}[\delta a_i^2] \leq C(\|q\|^2 \sigma_K^2 + A_0^3)$. Substituting this and the value factor into (17) and using $\sum_i a_i^2 \leq 1$,

$$\mathbb{E}[\|E_\times\|^2] \leq C(\|q\|^2 \sigma_K^2 + A_0^3) \|\mathbf{W}_O\|_F^2 \sigma_V^2. \quad (18)$$

Summarizing Step 2: in expectation the expansion (13) retains only the diagonal terms $\mathbb{E}[\|E_K\|^2]$ and $\mathbb{E}[\|E_V\|^2]$ together with a controlled remainder. The cross-terms (a) and (b) vanish exactly at all orders, while (c) and (d) are bounded by (16) and (18). Hence

$$\begin{aligned} \mathbb{E}[\|o - \hat{o}\|^2] &= \mathbb{E}[\|E_K\|^2] + \mathbb{E}[\|E_V\|^2] + R_2, \\ R_2 &:= \mathbb{E}[\|E_\times\|^2] + 2\mathbb{E}[E_V E_\times^\top] \\ &= \mathcal{O}\left((\|q\|^2 \sigma_K^2 + A_0^3) \|\mathbf{W}_O\|_F^2 \sigma_V^2\right). \end{aligned}$$

Step 3: The key distortion term $\mathbb{E}[\|E_K\|^2]$. We insert

the softmax expansion (11) into $E_K = \sum_i \delta a_i v_i \mathbf{W}_O$. By (12), $\sum_i \delta a_i = 0$, so for the constant vector o we have $\sum_i \delta a_i o = (\sum_i \delta a_i) o = 0$, subtracting this from E_K lets us recenter each $v_i \mathbf{W}_O$ by o without changing the value. Write $u_i := v_i \mathbf{W}_O - o$ and $D_o := \max_i \|u_i\|^2$. Grouping the softmax expansion as $\delta a_i = \delta a_i^{(1)} + \delta a_i^{(2)} + r_i$ Lemma 1,

$$E_K = \sum_{i=1}^T \delta a_i u_i = \sum_{i=1}^T (\delta a_i^{(1)} + \delta a_i^{(2)}) u_i + \underbrace{\sum_{i=1}^T r_i u_i}_{=: E_K^{(3+)}}.$$

Writing $E_K^{(m)} := \sum_i \delta a_i^{(m)} u_i$, so that $E_K = E_K^{(1)} + E_K^{(2)} + E_K^{(3+)}$ exactly with $E_K^{(3+)} := \sum_i r_i u_i$. We first evaluate the leading term exactly. Substituting $\delta a_i^{(1)} = a_i(\alpha_i - \bar{\alpha})$ and using $\sum_i a_i u_i = \sum_i a_i(v_i \mathbf{W}_O - o) = \sum_i a_i v_i \mathbf{W}_O - o = o - o = 0$ (the middle step holds by $\sum_i a_i = 1$), the mean-subtraction term drops out:

$$\begin{aligned} E_K^{(1)} &= \sum_i a_i(\alpha_i - \bar{\alpha}) u_i = \sum_i a_i \alpha_i u_i - \bar{\alpha} \underbrace{\sum_i a_i u_i}_{=0} \\ &= \sum_i a_i \alpha_i u_i. \end{aligned}$$

Therefore, using $\mathbb{E}[\alpha_i \alpha_\ell] = \delta_{i\ell} \mathbb{E}[\alpha_i^2]$,

$$\begin{aligned} \mathbb{E}[\|E_K^{(1)}\|^2] &= \sum_{i,\ell} a_i a_\ell \mathbb{E}[\alpha_i \alpha_\ell] u_i u_\ell^\top = \sum_i a_i^2 \mathbb{E}[\alpha_i^2] \|u_i\|^2 \\ &= \sum_{i,c} a_i^2 \|v_i \mathbf{W}_O - o\|^2 q_c^2 \mathbb{E}[(\delta k_{ic})^2], \end{aligned} \quad (19)$$

where the last step inserts $\mathbb{E}[\alpha_i^2] = \sum_c q_c^2 \mathbb{E}[(\delta k_{ic})^2]$ from (10). This is precisely D_K . In particular, by $\mathbb{E}[\alpha_i^2] \leq \|q\|^2 \sigma_K^2$ and $\sum_i a_i^2 \leq 1$, $(\mathbb{E}[\|E_K^{(1)}\|^2])^{1/2} \leq \|q\| \sigma_K D_o^{1/2} \leq A_0 D_o^{1/2}$.

Further we bound $E_K^{(3+)}$ via the per-token root-mean-square bound of Lemma 1. By the triangle inequality in L^2 , $\mathbb{E}[\|E_K^{(3+)}\|] \leq \max_i \|u_i\| \sum_i \mathbb{E}[|r_i|] \leq 7D_o^{1/2} A_0^3$, hence

$$(\mathbb{E}[\|E_K^{(3+)}\|^2])^{1/2} \leq 7D_o^{1/2} A_0^3. \quad (20)$$

Writing $E_K = E_K^{(1)} + E_K^{(2)} + E_K^{(3+)}$, the squared norm expands as

$$\mathbb{E}[\|E_K\|^2] = \mathbb{E}[\|E_K^{(1)}\|^2] + \underbrace{2\mathbb{E}[E_K^{(1)} E_K^{(2)\top}]}_{(\star)} + R_K, \quad (21)$$

$$\begin{aligned} R_K &:= \mathbb{E}[\|E_K^{(2)}\|^2] + 2\mathbb{E}[E_K^{(1)} E_K^{(3+)\top}] \\ &\quad + 2\mathbb{E}[E_K^{(2)} E_K^{(3+)\top}] + \mathbb{E}[\|E_K^{(3+)}\|^2]. \end{aligned} \quad (22)$$

The three factors have L^2 norms of orders A_0 , A_0^2 , and A_0^3 respectively: $(\mathbb{E}[\|E_K^{(1)}\|^2])^{1/2} \leq A_0 D_o^{1/2}$ by (19), $(\mathbb{E}[\|E_K^{(2)}\|^2])^{1/2} \leq \sum_i 2a_i A_0^2 \|u_i\| \leq 2A_0^2 D_o^{1/2}$ via the pathwise bound $|\delta a_i^{(2)}| \leq 2a_i A_0^2$ and $\sum_i a_i = 1$ and $(\mathbb{E}[\|E_K^{(3+)}\|^2])^{1/2} \leq 7A_0^3 D_o^{1/2}$ by (20). By Cauchy-Schwarz, each cross term in (22) is bounded by the product of the corresponding norms, so term by term:

- $\mathbb{E}[\|E_K^{(2)}\|^2] \leq 4D_o A_0^4$;
- $|2\mathbb{E}[E_K^{(1)} E_K^{(3+)\top}]| \leq 2(A_0 D_o^{1/2})(7A_0^3 D_o^{1/2}) = 14D_o A_0^4$
- $|2\mathbb{E}[E_K^{(2)} E_K^{(3+)\top}]| \leq 28D_o A_0^5$
- $\mathbb{E}[\|E_K^{(3+)}\|^2] \leq 49D_o A_0^6$

Combining the four bounds,

$$|R_K| \leq C D_o A_0^4 :$$

for $A_0 \leq 1$ directly, absorbing $A_0^5 \leq A_0^4$ and $A_0^6 \leq A_0^4$; for $A_0 > 1$ via the trivial estimate $\|E_K\| \leq \sum_i |\delta a_i| \|u_i\| \leq 2D_o^{1/2}$ (since $\sum_i |\delta a_i| \leq \sum_i (\hat{a}_i + a_i) = 2$), which together with the norm bounds above controls every term of (22) by $C D_o A_0^4$ in that regime. Now we evaluate the cross-term $(\star)$, since $E_K^{(1)} = \sum_i \delta a_i^{(1)} u_i$ is linear and $E_K^{(2)} = \sum_i \delta a_i^{(2)} u_i$ quadratic in $\{\alpha_\ell\}$, the scalar $E_K^{(1)} E_K^{(2)\top}$ is a homogeneous cubic in $\{\alpha_\ell\}$, and its expectation reduces to third-order moments $\mathbb{E}[\alpha_p \alpha_q \alpha_r]$. Each such moment falls into one of three exhaustive cases, all vanishing under Assumption 1. Three distinct indices are $\mathbb{E}[\alpha_p \alpha_q \alpha_r] = 0$ by mutual independence and zero mean, one repeated index $\mathbb{E}[\alpha_p^2 \alpha_q]$ with $p \neq q$ factors as $\mathbb{E}[\alpha_p^2] \mathbb{E}[\alpha_q] = 0$ also by independence and zero mean. A single cubed index, $\mathbb{E}[\alpha_p^3]$ vanishes by symmetry of α_p (established after (10)). Hence

$$(\star) = 2\mathbb{E}[E_K^{(1)} E_K^{(2)\top}] = 0, \quad (23)$$

so the second-order correction does not pollute the leading $\mathcal{O}(\|q\|^2 D_o \sigma_K^2)$ distortion, and all second-order and higher contributions are absorbed into the remainder.

Together with (23), the expansion (21) reads $\mathbb{E}[\|E_K\|^2] = D_K + R_K$ with $|R_K| \leq C D_o A_0^4$.

Step 4: The value distortion term $\mathbb{E}[\|E_V\|^2]$. The value term is exact and requires no expansion, as $E_V = \sum_i a_i \delta v_i \mathbf{W}_O$ with deterministic a_i . Thus

$$\mathbb{E}[\|E_V\|^2] = \sum_{i,\ell} a_i a_\ell \mathbb{E}[(\delta v_i \mathbf{W}_O)(\delta v_\ell \mathbf{W}_O)^\top].$$

By token independence and $\mathbb{E}[\delta v_i] = 0$, the $i \neq \ell$ terms vanish, leaving the diagonal. Expanding $\delta v_i \mathbf{W}_O = \sum_c \delta v_{ic} (\mathbf{W}_O)_{c,:}$ and using channel independence $\mathbb{E}[\delta v_{ic} \delta v_{i'c'}] = \delta_{cc'} \mathbb{E}[(\delta v_{ic})^2]$,

$$\begin{aligned} \mathbb{E}[\|E_V\|^2] &= \sum_i a_i^2 \mathbb{E}[\|\sum_c \delta v_{ic} (\mathbf{W}_O)_{c,:}\|^2] \\ &= \sum_i \sum_c a_i^2 \mathbb{E}[(\delta v_{ic})^2] \|\mathbf{W}_{O_c}\|_F^2, \end{aligned}$$

where $\mathbf{W}_{O_c} := (\mathbf{W}_O)_{c,:}$ is the c -th row of $\mathbf{W}_O$. This is precisely D_V .

Conclusion: Collecting Steps 2–4,

$$\mathbb{E}[\|o - \hat{o}\|^2] = \underbrace{\mathbb{E}[\|E_K^{(1)}\|^2]}_{D_K} + \underbrace{\mathbb{E}[\|E_V\|^2]}_{D_V} + \underbrace{R_2}_{2(c)+2(d)} + \underbrace{R_K}_{\text{Step 3}},$$

with $|R| = |R_2 + R_K| \leq C\|q\|^2 \|\mathbf{W}_O\|_F^2 \sigma_K^2 \sigma_V^2 + C(A_0^3 \|\mathbf{W}_O\|_F^2 \sigma_V^2 + D_o A_0^4)$. $\square$

For the above proof, we need a bound on the remainder. For this, we first record the derivatives of the softmax along the ray used in the proof.

Remark 6 (Softmax derivatives along the ray). *We record the derivatives of the softmax along the ray $\theta \mapsto s + \theta\alpha$, $\theta \in [0, 1]$, from which the expansion (11) follows by Taylor's theorem. With $a = \text{softmax}(s)$, write $\tilde{a}_\ell(\theta) := \text{softmax}(s + \theta\alpha)_\ell = a_\ell e^{\theta\alpha_\ell} / Z(\theta)$, $Z(\theta) := \sum_\ell a_\ell e^{\theta\alpha_\ell}$, and $\bar{\alpha}_\theta := \sum_\ell \tilde{a}_\ell(\theta) \alpha_\ell$, $\tilde{m}_2(\theta) := \sum_\ell \tilde{a}_\ell(\theta) (\alpha_\ell - \bar{\alpha}_\theta)^2$, $\tilde{m}_3(\theta) := \sum_\ell \tilde{a}_\ell(\theta) (\alpha_\ell - \bar{\alpha}_\theta)^3$ for the attention-weighted mean, variance, and third central moment of the perturbations under the intermediate weights. Then $\mathbf{g}(\theta) := \tilde{\mathbf{a}}_i(\theta)$ satisfies*

$$\begin{aligned} \mathbf{g}'(\theta) &= \tilde{\mathbf{a}}_i(\theta) (\alpha_i - \bar{\alpha}_\theta), \\ \mathbf{g}''(\theta) &= \tilde{\mathbf{a}}_i(\theta) [(\alpha_i - \bar{\alpha}_\theta)^2 - \tilde{m}_2(\theta)], \\ \mathbf{g}'''(\theta) &= \tilde{\mathbf{a}}_i(\theta) [(\alpha_i - \bar{\alpha}_\theta)^3 - 3(\alpha_i - \bar{\alpha}_\theta) \tilde{m}_2(\theta) - \tilde{m}_3(\theta)]. \end{aligned} \tag{24}$$

Indeed, differentiating $\log \tilde{a}_\ell(\theta) = \log a_\ell + \theta\alpha_\ell - \log Z(\theta)$ with $\frac{d}{d\theta} \log Z(\theta) = \bar{\alpha}_\theta$ gives $\tilde{a}'_\ell(\theta) = \tilde{a}_\ell(\theta) (\alpha_\ell - \bar{\alpha}_\theta)$; hence, using $\sum_\ell \tilde{a}_\ell(\theta) (\alpha_\ell - \bar{\alpha}_\theta) = 0$,

$$\begin{aligned} \bar{\alpha}'_\theta &= \sum_\ell \tilde{a}'_\ell(\theta) \alpha_\ell = \tilde{m}_2(\theta), \\ \tilde{m}'_2(\theta) &= \sum_\ell \tilde{a}'_\ell(\theta) (\alpha_\ell - \bar{\alpha}_\theta)^2 - 2\bar{\alpha}'_\theta \sum_\ell \tilde{a}_\ell(\theta) (\alpha_\ell - \bar{\alpha}_\theta) \\ &= \tilde{m}_3(\theta), \end{aligned}$$

and the three formulas follow the chain rule.

At $\theta = 0$ the weights reduce to the unperturbed a , so $\bar{\alpha}_0 = \bar{\alpha}$, $\tilde{m}_2(0) = \sum_\ell a_\ell (\alpha_\ell - \bar{\alpha})^2$, and $\mathbf{g}'(0) = a_i (\alpha_i - \bar{\alpha}) = \delta a_i^{(1)}$, $\frac{1}{2} \mathbf{g}''(0) = \frac{1}{2} a_i [(\alpha_i - \bar{\alpha})^2 - \sum_\ell a_\ell (\alpha_\ell - \bar{\alpha})^2] = \delta a_i^{(2)}$ the first- and second-order terms of (11) are exactly the first two Taylor coefficients of $\mathbf{g}$ at $\theta = 0$, and $\mathbf{r}_i$ is the exact second-order Taylor remainder of $\mathbf{g}$, as used in Lemma 1. Summing over i gives $\sum_i \delta a_i^{(1)} = \sum_i \delta a_i^{(2)} = 0$, consistent with (12).

Given the above remark, let us now state and prove the following lemma, providing the softmax tail bound.

Lemma 1 (Softmax tail bound). *Let $\mathbf{r}_i := \delta a_i - \delta a_i^{(1)} - \delta a_i^{(2)}$ denote the exact remainder of the softmax response past second order, as in (11). Under Assumptions 1 and 2, for every token i ,*

$$|\mathbf{r}_i| \leq 7 A_0^3 \quad \text{for every token } i \text{ and } \sum_i |\mathbf{r}_i| \leq 7 A_0^3 \tag{25}$$

and consequently, for any deterministic row vectors $\{u_i\}$,

$$\mathbb{E}[|\mathbf{r}_i|] \leq 7 A_0^3, \quad \sum_i \mathbb{E}[\mathbf{r}_i^2] \leq 49 A_0^6, \tag{26}$$

$$\left\| \sum_i \mathbf{r}_i u_i \right\| \leq 7 \max_i \|u_i\| A_0^3 \text{ a.s.}$$

Proof of Lemma 1. Fix a token i and set, for $\theta \in [0, 1]$,

$$\mathbf{g}(\theta) := \text{softmax}(s + \theta\alpha)_i,$$

$$\tilde{a}_\ell(\theta) := \text{softmax}(s + \theta\alpha)_\ell = \frac{a_\ell e^{\theta\alpha_\ell}}{Z(\theta)}, \quad Z(\theta) := \sum_\ell a_\ell e^{\theta\alpha_\ell},$$

and write $\bar{\alpha}_\theta := \sum_\ell \tilde{a}_\ell(\theta) \alpha_\ell$, $\tilde{m}_2(\theta) := \sum_\ell \tilde{a}_\ell(\theta) (\alpha_\ell - \bar{\alpha}_\theta)^2$, $\tilde{m}_3(\theta) := \sum_\ell \tilde{a}_\ell(\theta) (\alpha_\ell - \bar{\alpha}_\theta)^3$, the attention-weighted mean, variance, and third central moment of the perturbations under the intermediate weights. At $\theta = 0$ these reduce to $\bar{\alpha}$ and $\tilde{m}_2(0)$. By the ray derivatives recorded in Remark 6, equation (24),

$$\mathbf{g}'''(\theta) = \tilde{\mathbf{a}}_i(\theta) [(\alpha_i - \bar{\alpha}_\theta)^3 - 3(\alpha_i - \bar{\alpha}_\theta) \tilde{m}_2(\theta) - \tilde{m}_3(\theta)].$$

Since $\mathbf{g} \in C^3([0, 1])$ with $\mathbf{g}(0) = a_i$, $\mathbf{g}(1) = \hat{a}_i$, $\mathbf{g}'(0) = \delta a_i^{(1)}$, and $\frac{1}{2} \mathbf{g}''(0) = \delta a_i^{(2)}$, Taylor's theorem with integral remainder (see, e.g., [50]) gives the exact identity

$$\mathbf{r}_i = \frac{1}{2} \int_0^1 (1 - \theta)^2 \mathbf{g}'''(\theta) d\theta. \tag{27}$$

Now bound $\mathbf{g}'''$ pathwise. $|\alpha_\ell| = |q^\top \delta \mathbf{k}_\ell| \leq \sum_c |q_c| |\delta \mathbf{k}_{\ell c}| \leq A_0$ almost surely for every ℓ . Both $\bar{\alpha}_\theta$ and $\bar{\alpha}$ are convex combinations of the α_ℓ , so $|\bar{\alpha}_\theta| \leq A_0$ and $|\bar{\alpha}| \leq A_0$, hence $|\alpha_i - \bar{\alpha}_\theta| \leq 2A_0$, $\tilde{m}_2(\theta) \leq (2A_0)^2 = 4A_0^2$, $|\tilde{m}_3(\theta)| \leq (2A_0)^3 = 8A_0^3$, the last two because $\tilde{m}_2(\theta)$ and $\tilde{m}_3(\theta)$ are convex combinations of terms bounded by $4A_0^2$ and $8A_0^3$ respectively. The bracket in $\mathbf{g}'''(\theta)$ is therefore at most $8A_0^3 + 3 \cdot 2A_0 \cdot 4A_0^2 + 8A_0^3 = 40A_0^3$, uniformly in θ and in the token. For the prefactor, $\tilde{\mathbf{a}}_i(\theta)$ is a probability vector, so $\tilde{\mathbf{a}}_i(\theta) \leq 1$ per token and $\sum_i \tilde{\mathbf{a}}_i(\theta) = 1$ in aggregate.

Writing $\mathbf{g}_i$ for the response of token i , this gives, almost surely and uniformly in $\theta \in [0, 1]$,

$$|\mathbf{g}_i'''(\theta)| \leq 40A_0^3 \text{ and } \sum_i |\mathbf{g}_i'''(\theta)| \leq 40A_0^3 \sum_i \tilde{a}_i(\theta) = 40A_0^3.$$

Integrating (27) with $\int_0^1 (1-\theta)^2 d\theta = \frac{1}{3}$ yields both parts of (25): $|r_i| \leq \frac{20}{3}A_0^3 \leq 7A_0^3$, and $\sum_i |r_i| \leq \frac{1}{2} \int_0^1 (1-\theta)^2 \sum_i |\mathbf{g}_i'''(\theta)| d\theta \leq 7A_0^3$. The consequences (26) follow: an almost-sure bound dominates the mean $\sum_i \mathbb{E}[r_i^2] \leq \mathbb{E}[\max_i |r_i| \cdot \sum_i |r_i|] \leq 49A_0^6$ and $\|\sum_i r_i u_i\| \leq \max_i \|u_i\| \sum_i |r_i|$. Assumption 2 enters only through the pathwise bound $|\alpha_\ell| \leq A_0$. $\square$

B. Derivation of AATC via Waterfilling

Next, we derive the distortion of AATC given the transform from Section V-A. In the decorrelated basis the hypotheses of Theorem 1 holds by construction: each transformed channel is quantized independently by its own non-overloading $b_{\ell,c}$ -bit uniform quantizer, so the errors $\delta H_{ic} = \delta \hat{H}_{ic} - H_{ic}$ are zero-mean, symmetric, independent across tokens and channels, and bounded, with

$$\mathbb{E}[(\delta H_{ic})^2] = \frac{R_{\ell,c}^2}{12 \cdot 2^{2b_{\ell,c}}} = \gamma (\sigma_{\ell,c})^2 2^{-2b_{\ell,c}}, \quad (28)$$

where $\sigma_{\ell,c}$ is the standard deviation of transformed channel c (equal to $\sqrt{S_c}$ by the whitening), and γ absorbs the range-to-deviation ratio $R_{\ell,c} \propto \sigma_{\ell,c}$. Since γ is common to all channels, it drops out of the allocation. Instantiating Theorem 1 in this basis (effective query $\tilde{q} := \mathbf{B}^K q$, output metric $\tilde{M} := \mathbf{B}^V \mathbf{W}_O \mathbf{W}_O^\top (\mathbf{B}^V)^\top$) gives $D_K = \sum_{i,c} a_i^2 \|u_i\|^2 \tilde{q}_c^2 \mathbb{E}[(\delta H_{ic})^2]$ and $D_V = \sum_{i,c} a_i^2 \mathbb{E}[(\delta H_{ic})^2] \tilde{M}_{cc}$. Since attention weights are unavailable at compression time, we allocate against the attention-agnostic surrogate obtained by dropping the token-dependent factors and averaging $\tilde{q}_c^2$ over calibration queries:

$$D_K^\ell = \sum_{c=1}^{d_k} w_{\ell,c}^{(K)} (\sigma_{\ell,c}^{(K)})^2 \cdot 2^{-2b_{\ell,c}}, \quad (29)$$

with $w_{\ell,c}^{(K)} = \frac{1}{T} \sum_{i=1}^T ((\mathbf{B}_{\ell,c}^K)^\top q_{(i,\ell,c)})^2$

a per-channel weight with i being the tokens from the calibration set and T the size of the calibration set. The weight combines the query alignment and the reconstruction projection. $\sigma_{\ell,c}^{(K)}$ is the variance of the channel and $2^{-2b_{\ell,c}}$ captures the distortion at $b_{\ell,c}$ bits.

Remark 7. For grouped query attention in which multiple query heads attend to one key head we need to sum over all the queries that attend to this head in $w_{\ell,c}^{(K)}$, resulting in $w_{\ell,c}^{(K)} = \frac{1}{T} \sum_{q_{\text{head}}} \sum_{i=1}^T ((\mathbf{B}_{\ell,c}^K)^\top q_{(q_{\text{head}},i,\ell,c)})^2$

The value distortion takes the analogous form

$$D_V^\ell = \sum_{c=1}^{d_v} w_{\ell,c}^{(V)} (\sigma_{\ell,c}^{(V)})^2 \cdot 2^{-2b_{\ell,c}} \quad (30)$$

with $w_{\ell,c}^{(V)} = \|(\mathbf{B}^V \mathbf{W}_O)^\top\|_{\ell,c}^2$. This is the formula from Theorem 1 with the transform plugged into it. The above defines the distortions per-layer. Overall, we can obtain the global distortions as

$$D_K^{\text{global}} = \sum_{\ell=1}^L D_K^\ell; \quad D_V^{\text{global}} = \sum_{\ell=1}^L D_V^\ell.$$

Given the above, we can formulate the waterfilling problem as in Section V-B.

C. Mapping Related Work onto the Distortion Formula

1) *Token-Level Attention Weight*: The factor a_i^2 captures how strongly token i contributes to the attention output. Tokens with vanishing attention weights contribute negligibly to D no matter how their keys and values are quantized, so the natural way to exploit this is to drop such tokens entirely rather than spend bits on them. In the factorization of Eq. 5, a_i^2 sits inside the token-dependent factor, so methods that exploit it operate orthogonally to the channel-wise allocation we develop here. Token eviction methods take exactly this route.

The earliest of them use attention magnitude as the eviction criterion: StreamingLLM [10] preserves the first tokens together with a sliding window of recent tokens, H2O [9] retains "heavy hitter" tokens identified by accumulated attention scores, SnapKV [22] selects which tokens to keep during prefill from recent attention patterns. A more recent line of work refines this criterion with value-side information: VATP [23] take into account attention scores and value ℓ_1 -norms for token eviction, CriticalKV [24] introduces a method evicting tokens based on a projected value norm $\|V_i W_O\|_1$ combined with the attention scores and Expected Attention [26] uses $a_i \|W_O v_i\|$ as each KV pair's residual-stream contribution by approximating a_i through an expected attention score. All three target the contribution magnitude $a_i \|v_i W_O\|$, the size of token i 's direct contribution to the output. This ignores the attention mass that would redistribute to the remaining tokens if i were evicted. The full leading-order eviction cost includes that redistribution and takes the form $a_i \|v_i W_O - o\|$, which CAOTE [25] almost computes in closed form, will be discussed in Subsection C2.

2) *Token-Level Output Relevance*: The factor $\|v_i W_O - o\|^2$ measures how much token i 's value, after the output projection, deviates from the attention output. It arises in D_K as the sensitivity of the output to perturbations in attention allocation caused by key quantization. If $v_i W_O \approx o$, then key errors at this token are forgiving because moving attention onto or off token i does not change o much. Like a_i^2 , this factor lives in the token-dependent part of the factorization (Eq. 5) and would naturally belong to a token-wise allocation rather than the channel-wise one we address. The closest precedent in prior KV compression work is CAOTE [25], which defines token eviction error as the change in attention output upon token removal and derives in closed form that this error has the structure of a residual relative to the

attention-weighted average of the value vectors. CAOTE’s quantity differs from that in the attention-aware distortion formula by operating in value space rather than residual-stream space (i.e., before the output projection W_O). The structural insight is, however, shared. What matters is how different a token’s value contribution is from the rest of the output, not just how large its raw contribution is. The other value-aware eviction methods discussed in Section C1 instead target the contribution magnitude $a_i \|v_i W_O\|$, a related quantity that never references o and so misses this distinction.

3) *Query-Aware Precision*: The factor q_c^2 in D_K motivates allocating precision to the key directions that queries read from strongly. A²ATS [12] derives the attention score error component and uses it to motivate query-aware vector quantization. MixKVQ [21] uses query relevance for mixed-precision allocation. SQuat [20] projects the key quantization error onto the orthogonal complement of the query subspace, so that the retained error lies in directions queries do not read from; this is query-awareness through the error geometry rather than through bit allocation. OSCAR [19] addresses the factor through rotation: working with the full query covariance $C_Q = Q^\top Q$ rather than a per-channel energy, it shows under a frozen-error surrogate (post-rotation residual covariance) that the eigendecomposition of C_Q minimizes the surrogate (their Theorem 1). OSCAR composes this eigenrotation with a Hadamard transform and bit-reversal permutation, then applies uniform INT2 in the resulting basis. Our method takes a different route to the same goal on the key side. The rotation we use is obtained by whitening the cached activations and is not query-aware. The query signal enters through the bit allocation instead, with channels carrying more query energy receiving more bits. These two routes, query-awareness through the rotation versus through the allocation, coincide in their treatment of the query signal. Each, on its own, already directs precision toward the directions queries read from, so along that axis, applying both double-counts it. However, one could combine OSCAR’s query-aware rotation with channel-wise bit allocation.

4) *Output-Projection-Aware Value Precision*: The factor W_{O_c} , the column norm of the output projection at value channel c , appears in D_V as a multiplier on the value quantization error in that channel. A value channel that the output projection reads from with a large weight has its quantization error amplified at the output; a channel with a small projection weight is forgiving. The decomposition, therefore, motivates allocating value precision in proportion to W_{O_c} (or a norm-based aggregate over the output dimensions). To our knowledge, no prior method exposes W_{O_c} as an explicit allocation factor. W_O enters KVQuant’s Fisher sensitivity only implicitly, through the squared gradient of the language-modeling loss with respect to the value activations, $J(V) = \partial L / \partial V$, where W_O appears entangled with the attention scores and the rest of the downstream gradient. Our criterion instead names W_{O_c} as an explicit, closed-form structural

term. OSCAR’s value rotation uses the score-weighted value covariance $V^\top A^\top A V$, which contains no factor from the output projection. Our allocation criterion incorporates W_{O_c} directly, as a structural term derived from the decomposition which remains fixed for every layer, rather than a quantity inferred through calibration.

5) *Per-Channel Sensitivity*: The terms $\mathbb{E}[(\delta k_{ic})^2]$ and $\mathbb{E}[(\delta v_{ic})^2]$ capture how much quantization error each channel produces under a given quantizer. KIVI [5] addresses this by per-channel asymmetric scaling for keys and per-token scaling for values. KVQuant [6] additionally employs sensitivity-weighted non-uniform quantization. PALU [7] applies SVD-based low-rank truncation. RateQuant [11] is the closest to our approach. It applies rate-distortion theory to mixed-precision allocation, fitting a distortion-rate curve $D(b) = \alpha \beta^{-b}$ per component from calibration data and solving via reverse waterfilling. The substantive difference is not granularity (RateQuant operates at the (head, K-or-V) level but is granularity-agnostic in principle), but what the framework makes explicit. RateQuant’s sensitivity weights are squared gradient norms and its distortion curve is a parametric model whose parameters (α, β) are fit empirically from MSE at a handful of precisions. Query-awareness, the output-projection weights W_{O_c} and the attention-score weighting a_i never appear. They are bundled invisibly into the fitted gradient norms and curve parameters. Our decomposition surfaces these factors explicitly and ties each to a distinct design axis, deriving in closed form the structure that RateQuant leaves implicit in its empirical fits.

6) *Synthesis (Our Method)*: The four channel-level factors in the decomposition, namely q_c^2 and $\mathbb{E}[(\delta k_{ic})^2]$ on the key side as well as W_{O_c} and $\mathbb{E}[(\delta v_{ic})^2]$ on the value-side, together determine each channel’s contribution to D . Our method allocates non-uniform precisions across channels using a criterion that combines these factors. The rotation is calibration-derived from the cached activations and is shared across the bit-allocation step. We do not optimize it separately for query awareness, since the bit allocation already directs precision to the channels that matter most according to the attention statistics.

Each row of the table above is addressed by an existing method through an existing mechanism. Our contribution is to address the channel-level rows *jointly* through a single allocation criterion derived from the decomposition. Rather than picking one factor and optimizing for it via a single mechanism, we use the decomposition to set precisions that respond to all channel-level factors simultaneously. Our precise method is described in Section V.

D. Comparison of SVD and Whitening

KVTC [8] derives its transform from the PCA of $\mathbf{K} := \mathbf{X}\mathbf{W}$, whereas we use a whitening-based SVD of the projection $\mathbf{W}$. We show that these two constructions yield the same transform. The equivalence is confined to this decorrelation step: KVTC decorrelates a global cross-layer cache and allocates on reconstruction error, while

we decorrelate per layer and allocate on the output-aware distortion of Theorem 1.

Proposition 1 (Equivalence of Whitening-Based SVD and PCA of $\mathbf{XW}$). *Let $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{model}}}$ be calibration activations, $\mathbf{W} \in \mathbb{R}^{\text{model} \times d_k}$ a linear projection matrix, and define $\mathbf{K} := \mathbf{XW} \in \mathbb{R}^{n \times d_k}$. Let the empirical input covariance be $\Sigma_X := \frac{1}{n} \mathbf{X}^\top \mathbf{X}$ and assume Σ_X is symmetric positive definite. Define the whitened weight matrix $\widetilde{\mathbf{W}} := \Sigma_X^{-1/2} \mathbf{W}$. Let $\widetilde{\mathbf{W}} = \mathbf{USP}^\top$ be its singular value decomposition. Then the following statements hold:*

- 1) *The right singular vectors $\mathbf{P}$ coincide with the right singular vectors of $\mathbf{K}$.*
- 2) *The transform coding coordinates obtained via $\mathbf{KP}$ are identical to $\mathbf{X}\Sigma_X^{-1/2}\mathbf{US}$.*

Proof. First observe that

$$\mathbf{K}^\top \mathbf{K} = \mathbf{W}^\top \mathbf{X}^\top \mathbf{XW} = n \mathbf{W}^\top \Sigma_X \mathbf{W}.$$

On the other hand, $\widetilde{\mathbf{W}}^\top \widetilde{\mathbf{W}} = \mathbf{W}^\top \Sigma_X \mathbf{W}$. Therefore, $\mathbf{K}^\top \mathbf{K} = n \widetilde{\mathbf{W}}^\top \widetilde{\mathbf{W}}$. Since multiplication by the positive scalar n does not change eigenvectors, the right singular vectors of $\mathbf{K}$ are identical to those of $\widetilde{\mathbf{W}}$. Hence, both SVDs yield the same matrix $\mathbf{P}$ of right singular vectors.

Next, express $\mathbf{W}$ using the SVD of $\widetilde{\mathbf{W}}$: $\mathbf{W} = \Sigma_X^{-1/2} \mathbf{USP}^\top$. Substituting into $\mathbf{K} = \mathbf{XW}$ gives $\mathbf{K} = \mathbf{X}\Sigma_X^{-1/2} \mathbf{USP}^\top$. Right-multiplying by $\mathbf{P}$ yields $\mathbf{KP} = \mathbf{X}\Sigma_X^{-1/2} \mathbf{US}$. Thus the transform coding coordinates obtained by right-multiplying $\mathbf{K}$ with $\mathbf{P}$ are identical to those obtained via $\mathbf{X}\Sigma_X^{-1/2} \mathbf{US}$. $\square$

Remark 8. *First, the same holds for the values. Second, our method uses $\mathbf{U}\sqrt{\mathbf{S}}$ rather than $\mathbf{US}$, so even the transform-coding coordinates differ by the factor $\sqrt{\mathbf{S}}$, with $\mathbf{S}$ the singular values.*

E. Experimental Setting for KV Cache Quantization Methods

Let us explain in more detail the experimental settings we used for the different KV cache quantization methods appearing in Section VI.

a) *KIVI*: We use KIVI [5] as in its original implementation, quantizing keys channel-wise and values token-wise with recent-token window of size 128 and a scaling group-size of 32. We also keep the first 4 tokens in full precision.

b) *PALU*: For PALU [7], we apply their Fisher-uniform implementation with head group size 4 and the decompose-method whiten. For the low-rank part we establish a recent token window of size 128 with a sliding window of 16 and also keep the first 4 tokens in full precision. Further, we use uniform quantization with Hadamard rotation for the low-rank part.

c) *KVQuant*: We reimplemented KVQuant [6], following their original method. As in their work, we quantize keys per-channel and values per-token, quantize keys pre-RoPE, and use their non-uniform quantization using a

look-up table of 2^b signposts derived via Fisher-sensitivity-weighted k-means. We also implement their dense-and-sparse decomposition, which keeps the top 1% of magnitude outliers in full precision. For fairness, we match the FP16 window budget of the other methods, keeping a recent window of 128 tokens (flushed in blocks of 32) and keep the first 4 tokens in full precision.

d) *AATC*: If not stated otherwise, we keep the first 4 tokens in full precision and set the full precision recent token window to 128 with a sliding step 16. The bit range allowed for bit allocation per channel is 0 to 16 and the allocation is done globally across all layers for keys and values as described in Section V. Since the first three layers are quite important, but exhibit smaller activations and gradients and therefore smaller weights in the waterfilling problem, we protect them and give them the average amount of bits for the values and 1 bit more than average for the keys. We always use the same number of bits for keys and values. For the global allocation, we normalize σ^2 by each layer’s own 95-th percentile to remove the activation-scale bias between the layers, since early layers have smaller activations and thus smaller σ^2 and we do the same for $w^{(K)}$

e) *AATC var-only*: This refers to our method, but with allocating the bits via reverse waterfilling without using q_c^2 and W_{oc} in the weights, therefore only using the variance of the quantization error.

F. Benchmarks

Here is a more detailed explanation of the benchmarks we use in our experiments.

a) *LongBench v1 [40]*: is a bilingual, multi-task benchmark for evaluating long-context understanding, with document lengths ranging from roughly 1k to 34k tokens. We evaluate on seven English subsets spanning four task categories: single-document QA (TriviaQA, Qasper), query-based summarization (QMSum, SAMSum), few-shot classification (TREC), and code completion (LCC, RepoBench-P). Following the standard LongBench evaluation protocol, we use F1 score for QA tasks, ROUGE-L for summarization, accuracy for classification, and edit-similarity for code completion, and report results for each subset individually. We use greedy decoding with 0-shots and task-dependent maximum new tokens ranging from 32 to 512 (TriviaQA: 32, TREC: 64, Qasper: 128, SAMSum: 128, LCC: 64, RepoBench-P: 64, QMSum: 512).

b) *RULER [41] (What’s the Real Context Size of Your Long-Context Language Models?)*: also evaluates long context models and tests different context lengths to check if the model still performs well on longer contexts or when its performance degrades. The context lengths are split into 4k, 8k, 16k, 32k, 64k, and 128k, we focus on 4k-32k. It includes 13 tasks across 4 categories (retrieval, multi-hop tracing, aggregation, and question answering) with configurable sequence lengths (4k to 128k tokens). For evaluation, RULER uses task-specific metrics. For Needle-in-a-Haystack (NIAH), those are variable tracking, common

word extraction, and frequency word extraction, it checks whether all reference answers are found in the model’s response. Then it calculates $\text{Score} = (\text{samples where all answers are found}) / (\text{total samples}) \times 100$. For QA tasks, it checks if any reference answer is found in the model’s response and the score is calculated as $(\text{samples where at least one answer is found}) / (\text{total samples}) \times 100$. For evaluation, we use 500 samples per task, which is a common convention. RULER uses 0-shot greedy decoding with a maximum of 512 new tokens.

c) *GSM8K* (Grade School Math 8K) [42]: contains 8,500 linguistically diverse grade school math word problems that require multi-step reasoning. The problems take between 2 and 8 steps to solve and mostly involve basic algebra and arithmetic operations. The solutions are given in natural language. For evaluation, we report the percentage of answers correctly extracted from model outputs using 8-shot chain-of-thought prompting and the flexible-extract metric, which parses the final numerical answer from free-form generated text before comparison.

d) *MMLU-Pro* [44]: is a challenging multiple-choice benchmark extending MMLU with up to 10 answer options and questions requiring deeper reasoning. We evaluate on the math and computer science subjects only, using 5-shot prompting with examples drawn from the validation split of the same subject. The model is asked to respond with a single letter (A-J). We use greedy decoding with a 32-token budget and extract the predicted answer via regex matching on the generated text. Accuracy is reported as the fraction of test questions answered correctly across both subjects. LongBench uses greedy decoding, allows up to 32 new tokens and truncates the input to 4096 tokens.

e) *MATH-500*: [43] is a 500-problem subset of the MATH competition benchmark [51], spanning seven subjects (algebra, number theory, geometry, etc.) and five difficulty levels. We evaluate using 0-shot chain-of-thought prompting, instructing the model to reason step-by-step and place its final answer as `[answer]`. Answers are extracted from the last `[answer]` expression and graded for symbolic equivalence using math-verify, with a LaTeX normalization fallback. We report accuracy across all 500 problems. MATH-500 also uses greedy decoding, 2048 max new tokens.

G. Per-Subtask LongBench Results

Table III gives the per-subtask LongBench scores underlying the averages in Table II. The breakdown explains PALU’s weak average on Qwen. Its degradation is not uniform but almost entirely localized to the two code-completion tasks (RepoBench-P 0.34 vs. 0.67 for FP16, and LCC 0.48 vs. 0.61), while summarization is essentially lossless (QMSum 0.234 vs. 0.237). This is the characteristic signature of low-rank KV compression. The truncated low-rank components carry the fine-grained, token-level information that exact-match code completion and long-range retrieval depend on, whereas the retained dominant

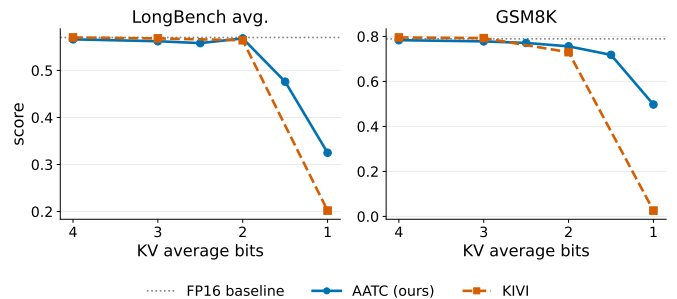


Fig. 2. LongBench average and GSM8K versus KV averages bits on Llama-3.1-8B-Instruct.

subspace suffices for short-context needle retrieval. The same mechanism accounts for PALU’s RULER profile in Table II, competitive at 4k but degrading monotonically with length.

H. Robustness across different Bit Budgets

Table V and Figure 2 sweep AATC across KV bit averages from 4-bit down to 1-bit. AATC degrades gracefully. The LongBench average remains within the noise of FP16 from 4-bit down to 2-bit, and GSM8K declines smoothly, with a pronounced drop appearing only at 1.5-bit and below. At the extreme 1-bit setting, AATC remains substantially more robust than KIVI (LongBench 0.33 vs. 0.20, GSM8K 0.50 vs. 0.03 on Llama), indicating that the transform-and-allocation design fails far more gracefully than fixed-precision quantization once the bit budget is exhausted.

Table IV isolates AATC’s 2-bit operating point ($\approx 7\times$) against the FP16 reference. The 2-bit quantization baselines (KIVI and KVQuant), which run at ≈ 5 to $6\times$, are reported in Table II. AATC stays close to FP16 on Qwen across all context lengths, while AATC var-only falls off on long-context RULER, consistent with Figure 1.

I. Ablation: Allocation and Calibration

Table VI ablates the two design axes of AATC at 2-bit on Llama. The bit-allocation strategy is the dominant factor. Replacing the adaptive allocation with a single global allocation costs 8.6 LongBench points (0.568 to 0.483), and removing the query-norm term (q_c^2) costs 3.8 points. In contrast, omitting the output-projection (W_O) term (-0.8) and using a per-layer rather than per-channel allocation (-1.6) does not drop performance significantly. For the calibration-corpus we cannot see a significant drop for only using OpenR1-Math -0.9 , for FineWeb the drop is slightly larger, which one could interpret as reasoning text being important for calibration. GSM8K is not much affected by any variants, so the ablation is resolved on LongBench. Consistent with Section G, the damage from the significant ablations concentrates on the precision-sensitive code and retrieval subtasks (global allocation drops LCC 0.63 to 0.47 and RepoBench-P 0.57 to 0.46).

TABLE III

PER-SUBTASK LONGBENCH SCORES AT THE MAIN-TABLE OPERATING POINT: F1 FOR QA (TRIVIAQA, QASPER), ROUGE-L FOR SUMMARIZATION (SAMSUM, QMSUM) AND QUERY SUMMARIZATION, ACCURACY FOR CLASSIFICATION (TREC), AND EDIT-SIMILARITY FOR CODE (LCC, REPOBENCH-P). **Bold** = BEST COMPRESSED METHOD PER COLUMN, AND THE FP16 BASELINE IS A REFERENCE.

	Method	TriviaQA	Qasper	TREC	SAMSum	LCC	RepoB-P	QMSum	avg.
Llama-3.1-8B-Instruct	Baseline	0.917	0.455	0.725	0.437	0.634	0.567	0.254	0.570
	KIVI	0.913	0.440	0.725	0.448	0.619	0.552	0.250	0.564
	PALU	0.892	0.449	0.710	0.438	0.539	0.576	0.248	0.550
	KVQuant	0.921	0.413	0.675	0.415	0.633	0.546	0.241	0.549
	AATC var	0.922	0.452	0.715	0.430	0.634	0.559	0.251	0.566
	AATC (ours)	0.919	0.444	0.690	0.434	0.623	0.546	0.252	0.558
Qwen-2.5-7B-Instruct	Baseline	0.900	0.434	0.715	0.460	0.606	0.668	0.237	0.574
	KIVI	0.597	0.187	0.568	0.337	0.283	0.316	0.168	0.351
	PALU	0.876	0.367	0.650	0.440	0.477	0.342	0.234	0.484
	KVQuant	0.900	0.409	0.700	0.449	0.583	0.640	0.226	0.558
	AATC var	0.896	0.425	0.675	0.461	0.569	0.648	0.233	0.558
	AATC (ours)	0.912	0.435	0.700	0.452	0.609	0.631	0.232	0.567

TABLE IV

AATC AT THE AGGRESSIVE 2-BIT OPERATING POINT ($\approx 7\times$ COMPRESSION), AGAINST THE FP16 REFERENCE. BASELINE COMPRESSION METHODS (KIVI, PALU, KVQUANT), WHICH RUN AT 2-BIT AND ≈ 5 TO $6\times$, ARE REPORTED IN THE MAIN RESULTS TABLE (II).

Method		Comp.	LongBench avg.	RULER				Reasoning			
				4k	8k	16k	32k	GSM8K	MMLU-m	MMLU-cs	MATH
Llama	Baseline (FP16)	1.00	0.570	0.859	0.856	0.828	0.775	0.789	0.438	0.456	0.454
	AATC var	7.0	0.543	0.818	0.822	0.775	0.677	0.767	0.402	0.427	0.484
	AATC (ours)	7.0	0.568	0.851	0.861	0.809	0.735	0.756	0.416	0.434	0.448
Qwen	Baseline (FP16)	1.00	0.574	0.720	0.716	0.718	0.720	0.818	0.708	0.567	0.680
	AATC var	6.9	0.518	0.688	0.653	0.640	0.566	0.795	0.644	0.510	0.664
	AATC (ours)	6.9	0.555	0.703	0.702	0.704	0.669	0.802	0.658	0.542	0.676

TABLE V

LONGBENCH PER-SUBTASK SCORES AND GSM8K ACROSS KV MULTIPLE BIT-BUDGETS. METRICS: F1 (TRIVIAQA, QASPER), ACCURACY (TREC), ROUGE-L (SAMSUM, QMSUM), EDIT-SIMILARITY (LCC, REPOBENCH-P). LB AVG. IS THE SEVEN-TASK MEAN.

	Config	Comp	TriviaQA	Qasper	TREC	SAMSum	LCC	RepoB-P	QMSum	LB avg.	GSM8K
Llama-3.1-8B-Instruct	Baseline FP16	1.0	0.917	0.455	0.725	0.437	0.634	0.567	0.254	0.570	0.789
	KIVI 4-bit	3.1	0.917	0.454	0.725	0.440	0.634	0.568	0.250	0.570	0.795
	KVQuant 4-bit	3.4	0.914	0.447	0.725	0.426	0.631	0.560	0.256	0.566	0.779
	AATC 4-bit	3.8	0.916	0.446	0.725	0.436	0.624	0.565	0.253	0.566	0.783
	KIVI 3-bit	3.8	0.914	0.450	0.725	0.435	0.634	0.564	0.254	0.568	0.792
	KVQuant 3-bit	4.3	0.921	0.452	0.725	0.433	0.631	0.559	0.250	0.567	0.776
	AATC 3-bit	4.9	0.920	0.458	0.705	0.439	0.613	0.549	0.251	0.562	0.778
	AATC-var 2-bit	7.0	0.897	0.435	0.650	0.436	0.619	0.519	0.243	0.543	0.767
	AATC 2-bit	7.0	0.917	0.456	0.720	0.438	0.630	0.570	0.248	0.568	0.756
	AATC 1.5-bit	9.1	0.874	0.336	0.630	0.434	0.480	0.358	0.223	0.476	0.718
	KIVI 1-bit	7.1	0.256	0.059	0.368	0.114	0.257	0.241	0.122	0.202	0.026
	AATC 1-bit	12.4	0.614	0.111	0.265	0.289	0.445	0.375	0.174	0.325	0.497
Qwen-2.5-7B-Instruct	Baseline FP16	1.0	0.900	0.434	0.715	0.460	0.606	0.668	0.237	0.574	0.818
	KIVI 4-bit	3.1	0.734	0.291	0.665	0.399	0.451	0.518	0.195	0.465	0.131
	KVQuant 4-bit	3.4	0.897	0.428	0.710	0.465	0.605	0.667	0.236	0.573	0.820
	AATC-var 4-bit	3.8	0.891	0.427	0.735	0.459	0.604	0.661	0.238	0.574	0.820
	AATC 4-bit	3.8	0.894	0.434	0.700	0.459	0.612	0.664	0.232	0.571	0.825
	KIVI 3-bit	3.8	0.686	0.233	0.620	0.374	0.334	0.380	0.188	0.402	0.182
	KVQuant 3-bit	4.3	0.921	0.452	0.725	0.433	0.631	0.558	0.250	0.567	0.816
	AATC-var 3-bit	4.9	0.907	0.424	0.705	0.464	0.596	0.652	0.233	0.569	0.832
	AATC 3-bit	4.9	0.888	0.433	0.705	0.457	0.599	0.667	0.238	0.570	0.818
	AATC-var 2-bit	6.9	0.871	0.365	0.620	0.445	0.500	0.603	0.222	0.518	0.795
	AATC 2-bit	6.9	0.898	0.386	0.705	0.456	0.597	0.612	0.230	0.555	0.802

TABLE VI

ABLATION OF AATC’S BIT-ALLOCATION AND CALIBRATION CHOICES (2-BIT, LLAMA-3.1-8B-INSTRUCT, WITH LONGBENCH PER-SUBTASK, LB AVG. = SEVEN-TASK MEAN AND GSM8K).

AATC variant (2-bit, Llama-3.1-8B-Instruct)	TriviaQA	Qasper	TREC	SAMSum	LCC	RepoB-P	QMSum	LB avg.	GSM8K
AATC 2-bit (standard)	0.917	0.456	0.720	0.438	0.630	0.570	0.248	0.568	0.751
<i>Allocation</i>									
no q_c^2 in allocation	0.910	0.410	0.685	0.444	0.545	0.473	0.248	0.531	0.752
no W_O in allocation	0.916	0.452	0.720	0.430	0.602	0.551	0.251	0.560	0.763
per-layer allocation	0.913	0.430	0.715	0.445	0.583	0.533	0.249	0.553	0.760
global allocation	0.847	0.337	0.595	0.432	0.475	0.462	0.231	0.483	0.740
<i>Calibration set</i>									
calib. OpenR1-Math only	0.915	0.449	0.710	0.454	0.600	0.540	0.248	0.559	0.761
calib. FineWeb only	0.895	0.437	0.690	0.446	0.564	0.540	0.248	0.546	0.754

REFERENCES

- [1] A. Gersho and R. M. Gray, *Vector quantization and signal compression*. USA: Kluwer Academic Publishers, 1991.
- [2] V. K. Goyal, "Theoretical foundations of transform coding," *IEEE Signal processing magazine*, vol. 18, no. 5, pp. 9–21, 2001.
- [3] T. M. Cover and J. A. Thomas, *Elements of information theory*. John Wiley & Sons, 2005.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [5] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "KIVI: A tuning-free asymmetric 2bit quantization for KV cache," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, Eds., vol. 235. PMLR, 21–27 Jul 2024, pp. 32 332–32 344.
- [6] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "Kvquant: Towards 10 million context length llm inference with kv cache quantization," *Advances in Neural Information Processing Systems*, vol. 37, pp. 1270–1303, 2024.
- [7] C.-C. Chang, W.-C. Lin, C.-Y. Lin, C.-Y. Chen, Y.-F. Hu, P.-S. Wang, N.-C. Huang, L. Ceze, M. Abdelfattah, and K.-C. Wu, "Palu: Kv-cache compression with low-rank projection," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 50 222–50 249.
- [8] K. Staniszeowski and A. Lancucki, "Kv cache transform coding for compact storage in llm inference," in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 37 768–37 815.
- [9] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett *et al.*, "H2o: Heavy-hitter oracle for efficient generative inference of large language models," *Advances in Neural Information Processing Systems*, vol. 36, pp. 34 661–34 710, 2023.
- [10] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 21 875–21 895.
- [11] F. Zuo, Z. Zhou, H. Cong, X. Xi, and H. F. Leung, "Rate-quant: Optimal mixed-precision kv cache quantization via rate-distortion theory," *arXiv preprint arXiv:2605.06675*, 2026.
- [12] J. He, J. Xing, N. Wang, R. Xu, S. Wu, P. Zhou, Q. Liu, C. J. Xue, and Q. Li, "A2ats: Retrieval-based kv cache reduction via windowed rotary position embedding and query-aware vector quantization," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 12 451–12 463.
- [13] J. Li, Y. Zhang, M. Y. Hassan, T. Chafekar, T. Cai, Z. Ren, P. Guo, B. Karimzadeh, C. J. Reed, C. Wang, and C. Gan, "Commvq: commutative vector quantization for kv cache compression," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. ICML'25, 2025.
- [14] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni, "Turbo-quant: Online vector quantization with near-optimal distortion rate," in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 56 418–56 439.
- [15] U. Saxena, G. Saha, S. Choudhary, and K. Roy, "Eigen attention: Attention in low-rank space for kv cache compression," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 15 332–15 344.
- [16] C.-C. Chang, W.-C. Lin, C.-Y. Lin, H.-Y. Chiang, Y. Akhauri, X. Dai, H. Jiang, Y. Li, L. Ceze, K.-C. Wu, and M. S. Abdelfattah, "xKV: Cross-layer KV-cache compression via aligned singular vector extraction," in *Forty-third International Conference on Machine Learning*, 2026.
- [17] A. Liu, J. Liu, Z. Pan, Y. He, R. Haffari, and B. Zhuang, "Minicache: Kv cache compression in depth dimension for large language models," *Advances in Neural Information Processing Systems*, vol. 37, pp. 139 997–140 031, 2024.
- [18] Z. Su, Z. Chen, W. Shen, H. Wei, L. Li, H. Yu, and K. Yuan, "Rotatkv: Accurate and robust 2-bit kv cache quantization for llms via outlier-aware adaptive rotations," *arXiv preprint arXiv:2501.16383*, 2025.
- [19] Z. Zhou, D. Zhuang, J. Li, Z. Chen, S. L. Song, B. Athiwaratkun, and X. Wu, "Oscar: Offline spectral covariance-aware rotation for 2-bit kv cache quantization," *arXiv preprint arXiv:2605.17757*, 2026.
- [20] H. Wang, L. Han, K. Xu, and A. Srivastava, "SQat: Subspace-orthogonal KV cache quantization," in *Second Conference on Language Modeling*, 2025.
- [21] T. Zhang, Z. Zeng, H. Peng, H. Zhuang, and C. Chen, "Mixkvq: Query-aware mixed-precision kv cache quantization for long-context reasoning," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026, pp. 7189–7204.
- [22] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, "Snapkv: Llm knows what you are looking for before generation," *Advances in Neural Information Processing Systems*, vol. 37, pp. 22 947–22 970, 2024.
- [23] Z. Guo, H. Kamigaito, and T. Watanabe, "Attention score is not all you need for token importance indicator in kv cache reduction: Value also matters," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 21 158–21 166.
- [24] Y. Feng, J. Lv, H. Guo, Y. Cao, S. K. Zhou, and X. Xie, "Criticallykv: Optimizing kv cache eviction from an output perturbation perspective," *arXiv preprint arXiv:2502.03805*, 2025.
- [25] R. Goel, J. Park, M. Gagrani, D. Jones, M. J. Morse, M. H. Langston, C. Lott, and M. Lee, "Caote: Optimizing kv cache memory through attention output error-based token eviction," in *ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems*, 2026.
- [26] A. Devoto, M. Jeblick, and S. Jégou, "Expected attention: Kv cache compression by estimating attention from future queries distribution," *arXiv preprint arXiv:2510.00636*, 2025.
- [27] W. Cheng, S. Dong, J. Qin, and W. Wang, "Qaq: Quality adaptive quantization for llm kv cache," in *2025 IEEE/CVF International Conference on Computer Vision Workshops (IC-CVW)*. IEEE, 2025, pp. 2563–2571.
- [28] D. Calver, "Runtime-certified bounded-error quantized attention," *arXiv preprint arXiv:2605.20868*, 2026.
- [29] P.-C. Su, "eoptshrinkq: Near-lossless kv cache compression through optimal spectral denoising and quantization," *arXiv preprint arXiv:2605.02905*, 2026.
- [30] J. Huang and P. Schultheiss, "Block quantization of correlated gaussian random variables," *IEEE Transactions on Communications Systems*, vol. 11, no. 3, pp. 289–296, 1963.
- [31] N. Jayant, J. Johnston, and R. Safranek, "Signal compression based on models of human perception," *Proceedings of the IEEE*, vol. 81, no. 10, pp. 1385–1422, 1993.
- [32] J. Ballé, P. A. Chou, D. Minnen, S. Singh, N. Johnston, E. Agustsson, S. J. Hwang, and G. Toderici, "Nonlinear transform coding," *IEEE Journal of Selected Topics in Signal Processing*, vol. 15, no. 2, pp. 339–353, 2020.
- [33] G. K. Wallace, "The jpeg still picture compression standard," *Communications of the ACM*, vol. 34, no. 4, pp. 30–44, 1991.
- [34] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever *et al.*, "Improving language understanding by generative pre-training," *Technical report, OpenAI*, 2018.
- [35] J. Ainslie, J. Lee-Thorp, M. De Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "Gqa: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of the 2023 conference on empirical methods in natural language processing*, 2023, pp. 4895–4901.
- [36] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. Van Baalen, and T. Blankevoort, "A white paper on neural network quantization," *arXiv preprint arXiv:2106.08295*, 2021.
- [37] W. R. Bennett, "Spectra of quantized signals," *The Bell System Technical Journal*, vol. 27, no. 3, pp. 446–472, 1948.
- [38] B. Widrow, I. Kollar, and M.-C. Liu, "Statistical theory of quantization," *IEEE Transactions on instrumentation and measurement*, vol. 45, no. 2, pp. 353–361, 1996.
- [39] R. Gray and D. Neuhoff, "Quantization," *IEEE Transactions on Information Theory*, vol. 44, no. 6, pp. 2325–2383, 1998.
- [40] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou *et al.*, "Longbench: A bilingual, multi-task benchmark for long context understanding," in *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, 2024, pp. 3119–3137.

- [41] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, and B. Ginsburg, “RULER: What’s the real context size of your long-context language models?” in *First Conference on Language Modeling*, 2024.
- [42] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano *et al.*, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [43] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, “Let’s verify step by step,” in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 39 578–39 601.
- [44] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang *et al.*, “Mmlu-pro: A more robust and challenging multi-task language understanding benchmark,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 95 266–95 290, 2024.
- [45] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [46] Q. A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, G. Dong, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y.-C. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, Z. Qiu, S. Quan, and Z. Wang, “Qwen2.5 technical report,” *ArXiv*, vol. abs/2412.15115, 2024.
- [47] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” *arXiv preprint arXiv:1609.07843*, 2016.
- [48] G. Penedo, H. Kydlicek, A. Lozhkov, M. Mitchell, C. Raffel, L. Von Werra, T. Wolf *et al.*, “The fineweb datasets: Decanting the web for the finest text data at scale,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 30 811–30 849, 2024.
- [49] Hugging Face, “Open r1: A fully open reproduction of deepseek-r1,” January 2025. [Online]. Available: <https://github.com/huggingface/open-r1>
- [50] V. A. Zorich, *Mathematical Analysis I*. Springer, 2004.
- [51] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, “Measuring mathematical problem solving with the math dataset,” *NeurIPS*, 2021.